

# 数据与算法 - 第6讲

## 树、二叉树 (二)

冯伟 ([fengwei@tsinghua.edu.cn](mailto:fengwei@tsinghua.edu.cn))

2024年10月22日



# 提纲



- 二叉搜索树
- 霍夫曼树与霍夫曼编码
- 堆



- 二叉搜索树
- 霍夫曼树与霍夫曼编码
- 堆

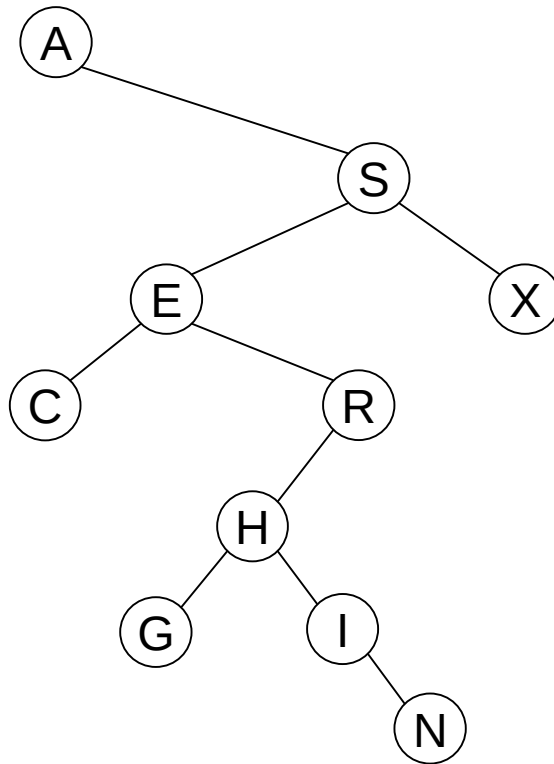


# 二叉搜索树 - 基本概念

- 线性表中查找一个特定元素的复杂度是 $O(N)$
- 设计新的数据结构以进一步提高查找效率
- 二叉搜索树的效率可达 $O(\log N)$
- 二叉搜索树 (Binary Search Tree, BST) 或者是一棵空树，或者是具有下列性质的二叉树：
  - 每个结点有一个关键字 (key)
  - 任意结点关键字大于等于该结点左子树中所有结点关键字
  - 任意结点关键字小于等于该结点右子树中所有结点关键字
- 虽然二叉搜索树是非线性结构，但却隐含了一个线性有序序列
  - 中序遍历可以得到一个有序序列



# 二叉搜索树的例子



中序遍历：A - C - E - G - H - I - N - R - S - X



# 二叉搜索树 - 实现

```
typedef struct Item {                                //数据项定义
    int m_Key;                                       //关键字
    float info;
} ITEM;

template<class Item> struct BST_Node {               //结点定义
    Item item;
    BST_Node<Item> *left, *right;                   //左右孩子结点指针
    BST_Node(Item x)                                //结点构造函数
        {item = x, left = NULL; right = NULL;}
};

typedef BST_Node<Item> *Link;                        //结点指针类型定义
```



# 二叉搜索树 - 实现

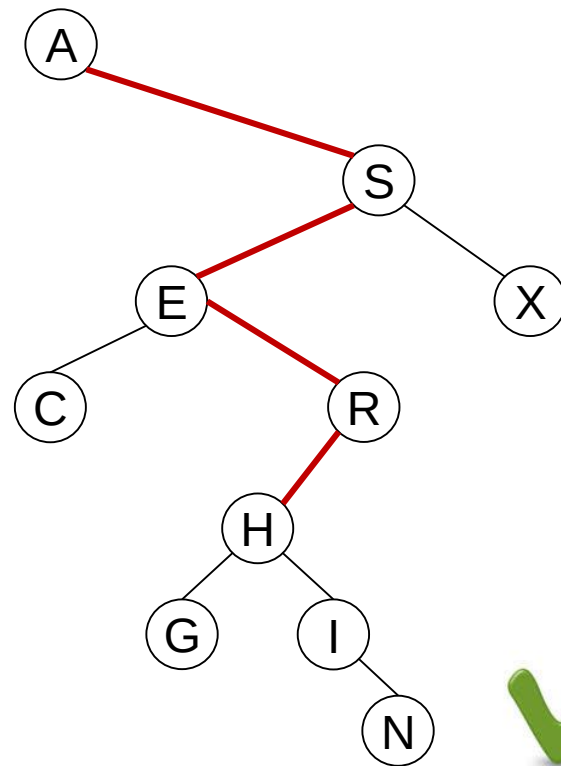
```
template<class Item>class BST{  
private:  
    Link  head;                                //BST的头结点  
    Item  searchR(Link p, int &x);             //查找函数  
    void  insertR(Link& p, Item x);            //插入函数  
public:  
    BST() {head = 0;}                          //构造函数  
    inline Item search(int &x) {return searchR(head, x);} //调用查找函数  
    inline void insert(Item item) {insertR(head,item);}  //调用插入函数  
};
```



# 二叉搜索树 - 查找

- 对于非空二叉树，查找结点值为item，则：
  - 若当前结点值为item，则查找成功，返回结点值，返回真
  - 若当前结点值大于item，进入左子树中查找
  - 若当前结点值小于item，进入右子树中查找
- 可以用递归算法实现

例子：在BST中查找H



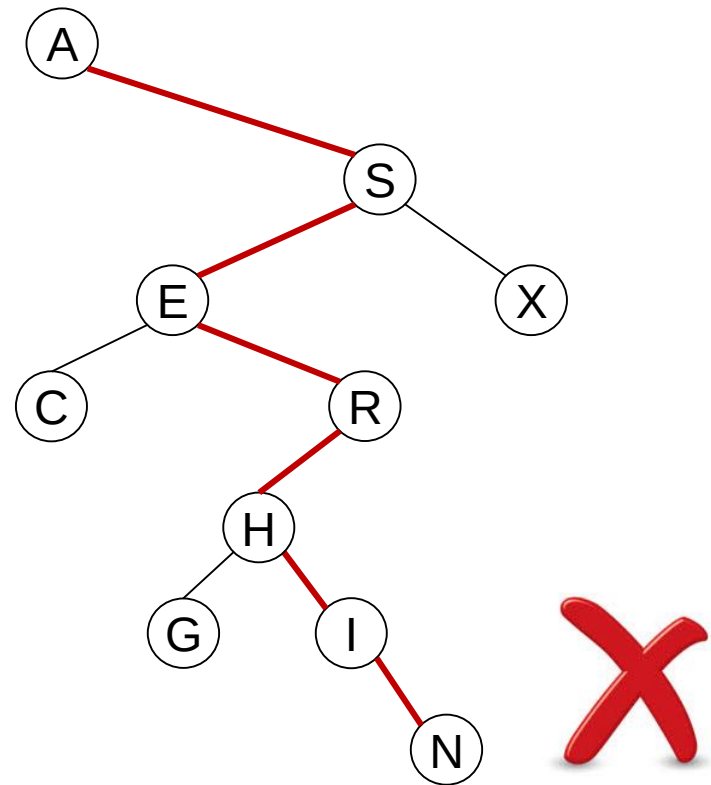




# 二叉搜索树 - 查找

- 对于非空二叉树，查找结点值为item，则：
  - 若当前结点值为item，则查找成功，返回结点值，返回真
  - 若当前结点值大于item，进入左子树中查找
  - 若当前结点值小于item，进入右子树中查找
- 可以用递归算法实现

例子：在BST中查找M





# 二叉搜索树 - 查找

```
template<class Item>
```

```
Item BST<Item>::searchR(Link p, int &x){
```

```
    if(!p) return NULL;
```

//搜索失败

```
    int key = p->item.m_Key;
```

//取关键字

```
    if(x == key) return p->item;
```

//搜索成功

```
    if(x < key)
```

//小于当前结点

```
        return searchR(p->left, x);
```

//搜索左子树

```
    else
```

//大于当前结点

```
        return searchR(p->right, x);
```

//搜索右子树

```
}
```



# 二叉搜索树 - 查找

- 二叉搜索树查找算法的时间复杂度与树的形态有关，查找次数不超过其深度（走一步进一层）
  - 理想平衡树： $O(\log_2 N)$
  - 单支树： $O(N)$
  - 平均： $O(\log_2 N)$
- 空间复杂度（递归栈）类似
- 构造平衡的二叉树很重要



# 二叉搜索树 - 插入

- 插入与查找过程类似：查找失败后（找到了位置）执行插入操作
- 插入算法本质上是查找操作，所以其时间复杂度和空间复杂度与查找算法相同



# 二叉搜索树 - 插入

```
template<class Item>
```

```
void BST<Item>::insertR(Link& p, Item item){
```

```
    if(!p) {
```

//插入头结点

```
        p = new BST_Node<Item>(item); return;
```

```
    }
```

```
    if(item.m_Key < p->item.m_Key)
```

//小于结点关键字

```
        insertR(p->left, item);
```

//在左子树插入

```
    else
```

//大于结点关键字

```
        insertR(p->right, item);
```

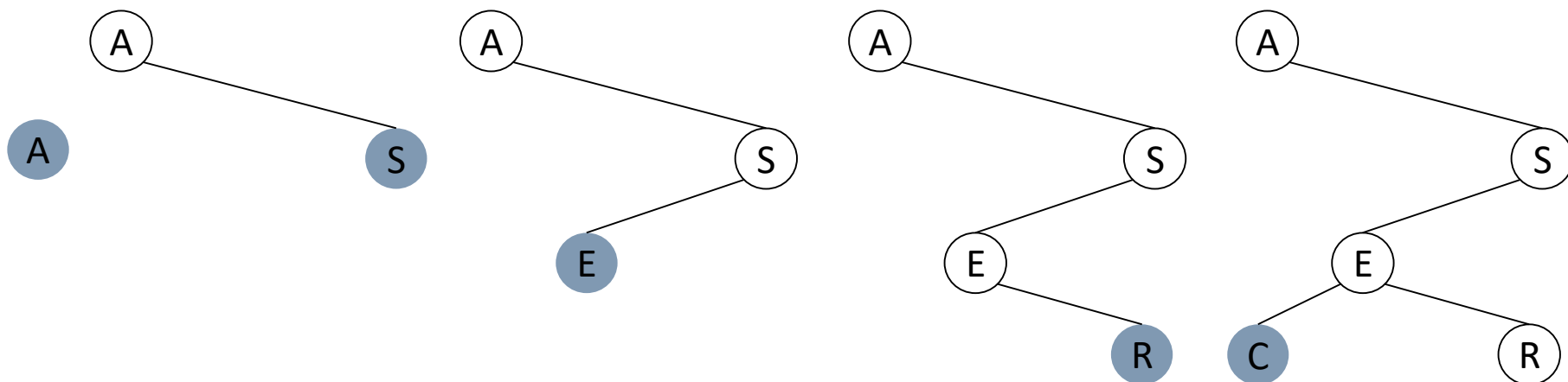
//在右子树插入

```
}
```



# 二叉搜索树 - 构造

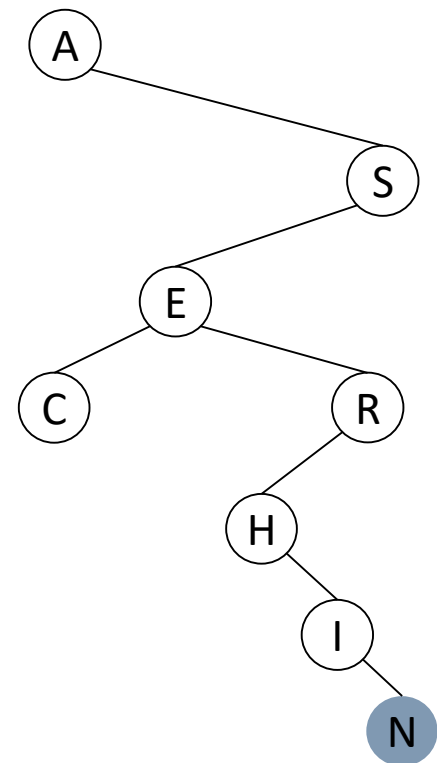
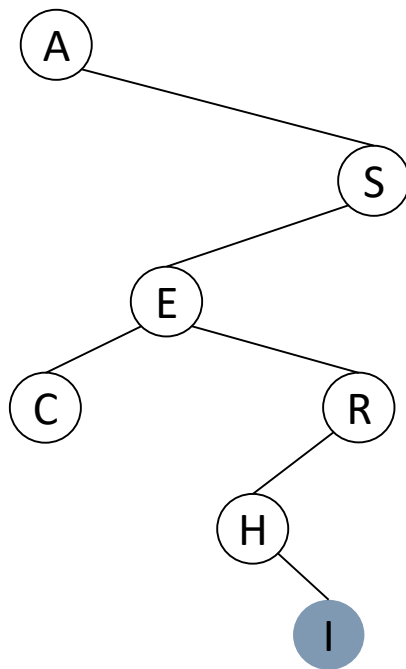
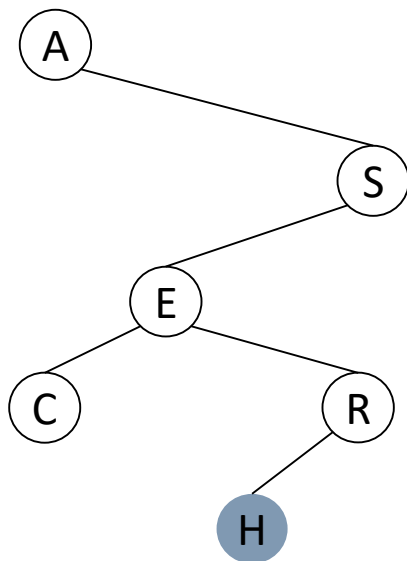
- 重复调用插入算法，可以构造二叉搜索树
- 例子：给定关键字序列 A S E R C H I N，构造一棵二叉搜索树





# 二叉搜索树 - 构造

- 重复调用插入算法，可以建立二叉搜索树
- 例子：给定关键字序列 A S E R C H I N，构造一棵二叉搜索树



# 提纲



- 二叉搜索树
- 霍夫曼树与霍夫曼编码
- 堆

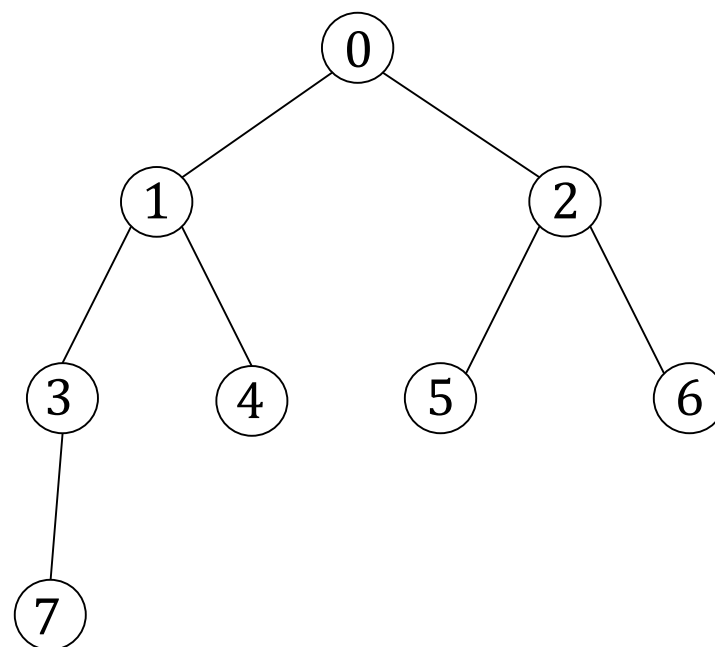




# 霍夫曼树 - 基本概念

霍夫曼树又称为**最优二叉树**，可以用于解决**最优编码问题**

- 结点到结点的分支称为**路径**
- 路径上的分支数称为**路径长度**，记为 $l$
- **结点的路径长度**定义为从根结点到该结点的路径上分支的数目
- **树的路径长度**是所有叶子结点的路径长度之和，记为 $L$ 。



$$L = 3 \times 1 + 2 \times 3 = 9$$



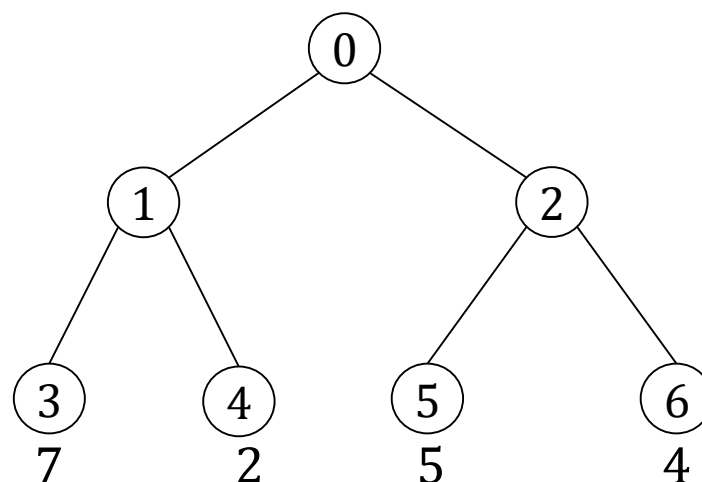
# 霍夫曼树 - 基本概念

一些基本概念（请同学们思考其中定性、定量的关系）

➤ 对分支加权的路径长度称为带权路径长度

➤ 树的带权路径长度为树中所有叶子结点的带权路径长度之和，记为  $WPL$

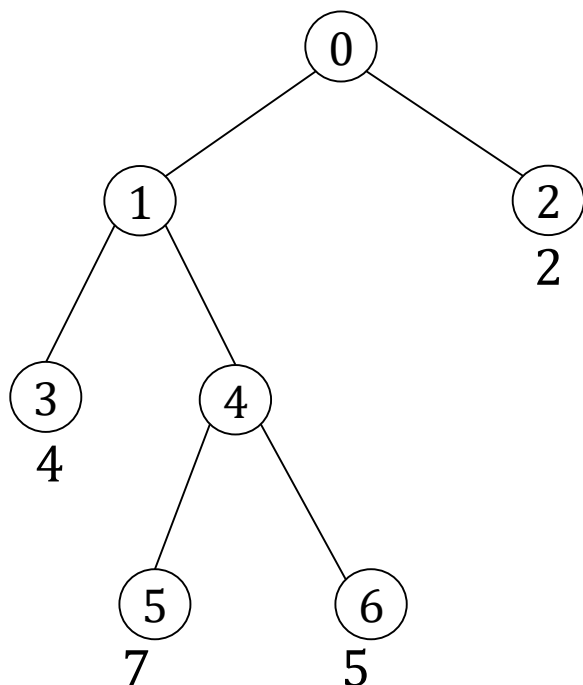
➤  $WPL = \sum_{i=1}^n W_i P_i$



$$WPL = 7 \times 2 + 2 \times 2 + 5 \times 2 + 4 \times 2 = 36$$

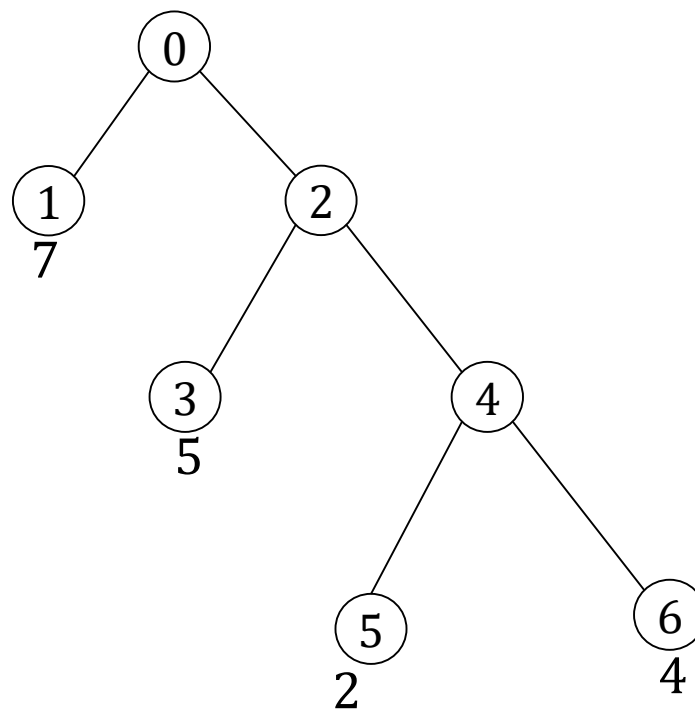


# 两个例子



$$L = 2 + 3 + 3 + 1 = 9$$

$$WPL = 4 \times 2 + 7 \times 3 + 5 \times 3 + 2 \times 1 = 46$$



$$L = 1 + 2 + 3 + 3 = 9$$

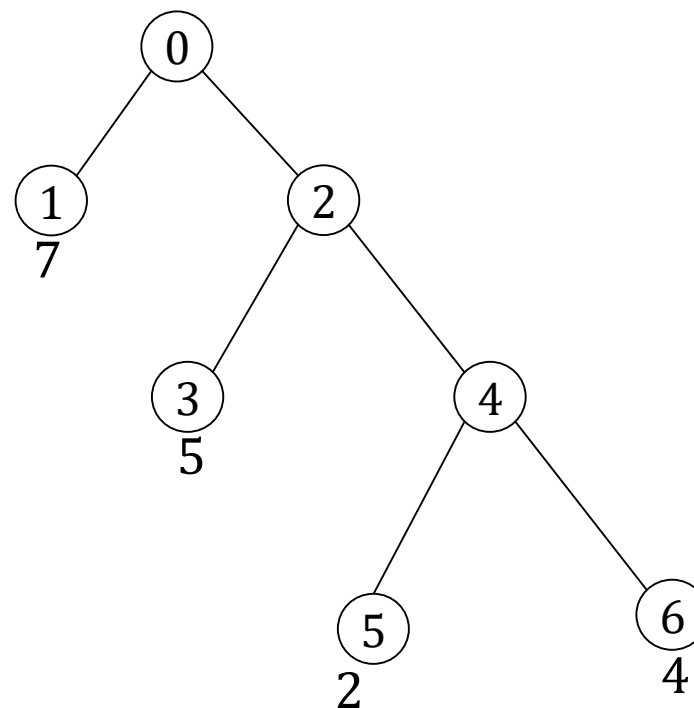
$$WPL = 7 \times 1 + 5 \times 2 + 2 \times 3 + 4 \times 3 = 35$$

具有相同路径长度和不同加权路径长度的两颗二叉树



# 霍夫曼树 - 基本概念

- 在所有路径长度相同的二叉树中， $WPL$ 最小的树称为最优二叉树，或霍夫曼树 (Huffman Tree)
- 直观来看，在霍夫曼树中，权值大的结点离根最近，权值小的结点离根结点远





# 霍夫曼树的构造

## 霍夫曼算法：构造最优二叉树的算法

- 根据给定的 $n$ 个权值 $\{w_1, w_2 \cdots w_n\}$ ，构造一棵二叉树
- 二叉树的 $n$ 个叶子结点的权值分别为 $w_1, w_2 \cdots w_n$
- 树中只有度为0和度为2的结点



# 霍夫曼树的构造

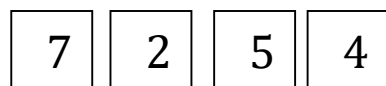
1. 根据给定的 $n$ 个权值 $\{w_1, w_2 \cdots w_n\}$ ，构造 $n$ 棵二叉树的集合 $F = \{T_1, T_2 \cdots T_n\}$ ，其中每棵二叉树中均只含一个带权值为 $w_i$ 的根结点，其左、右子树为空树
2. 在 $F$ 中选取其根结点的权值为最小的两棵二叉树，分别作为左、右子树构造一棵新的二叉树，并置这棵新的二叉树根结点的权值为其左、右子树根结点的权值之和
3. 从 $F$ 中删去这两棵树，同时加入刚生成的新树
4. 重复2和3两步，直至 $F$ 中只含一棵树为止



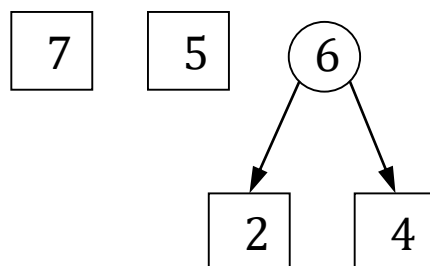
# 霍夫曼树的构造

一个例子：（具体代码实现参考教材90-91页）

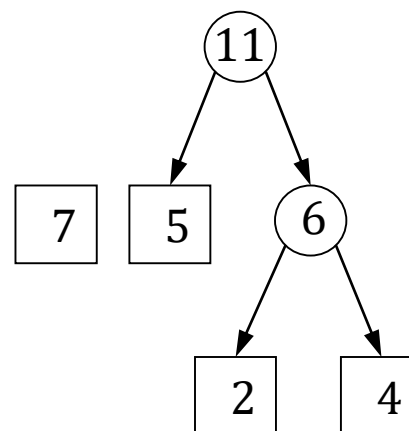
初始  
 $F = \{7, 2, 5, 4\}$



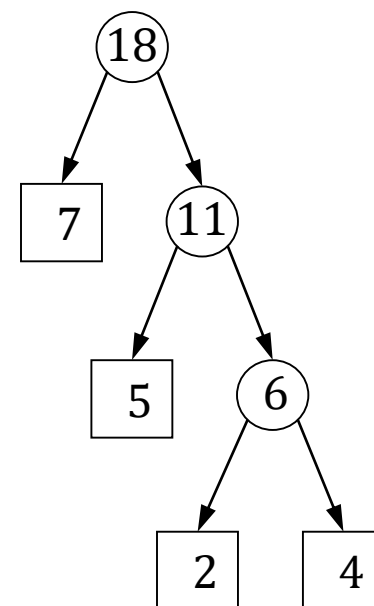
合并2, 4  
 $F = \{7, 5, 6\}$



合并5, 6  
 $F = \{7, 11\}$



合并7, 11  
 $F = \{18\}$





# 霍夫曼树的应用：最优编码问题

- 给定一个需要编码的字符集  $S = \{s_1, s_2, \dots, s_n\}$ ，同时给定  $s_i$  在文本中的出现概率  $p_i$ ，满足  $\sum_{i=1}^n p_i = 1$
- 编码：将文本转换为0或者1组成的序列
- 要求：
  - 可以通过译码算法恢复原始文本
  - 表示原始文本的总编码长度尽可能短





# 定长编码

- 字符集  $S = \{a, b, c, d, e\}$ 
  - 将文本编码成0/1序列
  - 由数字序列可以直接译码原始文本
- 字符出现的概率为
  - $\{0.12, 0.40, 0.15, 0.08, 0.25\}$
  - $\sum_{i=1}^n p_i = 1$
- 平均码长  $\sum_{i=1}^5 l_i p_i = l \sum_{i=1}^5 p_i = l = 3$
- 不存在比该方案更短的定长编码方案

| 字符  | 概率   | 码字  |
|-----|------|-----|
| $a$ | 0.12 | 000 |
| $b$ | 0.40 | 001 |
| $c$ | 0.15 | 010 |
| $d$ | 0.08 | 011 |
| $e$ | 0.25 | 100 |

001,010,011 译码   $b\ c\ d$



# 定长编码的例子：ASCII码

- 在计算机系统中，ASCII码很常用，是一种定长编码
- 将字母、数字和其它符号编码成7位二进制表示的整数

| b <sub>7</sub> b <sub>6</sub> b <sub>5</sub>                |   | 000 | 001 | 010          | 011         | 100      | 101      | 110      | 111      |
|-------------------------------------------------------------|---|-----|-----|--------------|-------------|----------|----------|----------|----------|
| b <sub>4</sub> b <sub>3</sub> b <sub>2</sub> b <sub>1</sub> |   | 0   | 1   | 2            | 3           | 4        | 5        | 6        | 7        |
| 0000                                                        | 0 | NUL | DLE | <b>SP</b>    | <b>0</b>    | <b>@</b> | <b>P</b> | <b>`</b> | <b>p</b> |
| 0001                                                        | 1 | SOH | DC1 | <b>!</b>     | <b>1</b>    | <b>A</b> | <b>Q</b> | <b>a</b> | <b>q</b> |
| 0010                                                        | 2 | STX | DC2 | <b>"</b>     | <b>2</b>    | <b>B</b> | <b>R</b> | <b>b</b> | <b>r</b> |
| 0011                                                        | 3 | ETX | DC3 | <b>#</b>     | <b>3</b>    | <b>C</b> | <b>S</b> | <b>c</b> | <b>s</b> |
| 0100                                                        | 4 | EOT | DC4 | <b>\$</b>    | <b>4</b>    | <b>D</b> | <b>T</b> | <b>d</b> | <b>t</b> |
| 0101                                                        | 5 | ENQ | NAK | <b>%</b>     | <b>5</b>    | <b>E</b> | <b>U</b> | <b>e</b> | <b>u</b> |
| 0110                                                        | 6 | ACK | SYN | <b>&amp;</b> | <b>6</b>    | <b>F</b> | <b>V</b> | <b>f</b> | <b>v</b> |
| 0111                                                        | 7 | BEL | ETB | <b>'</b>     | <b>7</b>    | <b>G</b> | <b>W</b> | <b>g</b> | <b>w</b> |
| 1000                                                        | 8 | BS  | CAN | <b>(</b>     | <b>8</b>    | <b>H</b> | <b>X</b> | <b>h</b> | <b>x</b> |
| 1001                                                        | 9 | HT  | EM  | <b>)</b>     | <b>9</b>    | <b>I</b> | <b>Y</b> | <b>i</b> | <b>y</b> |
| 1010                                                        | A | LF  | SUB | <b>*</b>     | <b>:</b>    | <b>J</b> | <b>Z</b> | <b>j</b> | <b>z</b> |
| 1011                                                        | B | VT  | ESC | <b>+</b>     | <b>;</b>    | <b>K</b> | <b>[</b> | <b>k</b> | <b>{</b> |
| 1100                                                        | C | FF  | FS  | <b>,</b>     | <b>&lt;</b> | <b>L</b> | <b>\</b> | <b>l</b> | <b> </b> |
| 1101                                                        | D | CR  | GS  | <b>-</b>     | <b>=</b>    | <b>M</b> | <b>]</b> | <b>m</b> | <b>}</b> |
| 1110                                                        | E | SO  | RS  | <b>.</b>     | <b>&gt;</b> | <b>N</b> | <b>^</b> | <b>n</b> | <b>~</b> |
| 1111                                                        | F | SI  | US  | <b>/</b>     | <b>?</b>    | <b>O</b> | <b>_</b> | <b>o</b> | DEL      |



# 变长编码

- 变长编码：出现概率高的字符以较短码长进行编码，出现概率低的以较长码长进行编码，从而缩短码长

- 按概率顺序

$$b > e > c > a > d$$

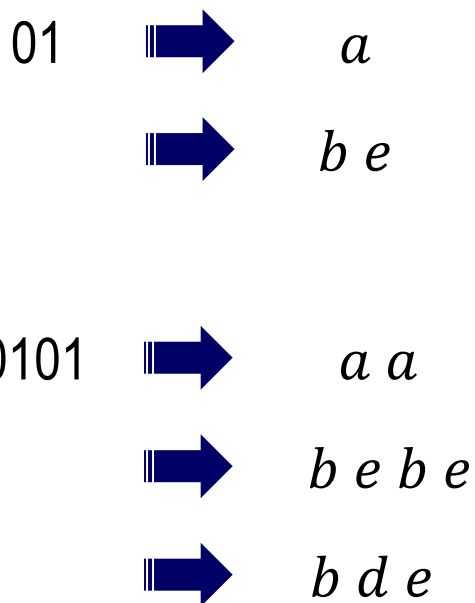
- 平均码长

$$\sum_{i=1}^5 l_i p_i = 1 \times (0.40 + 0.25) + 2 \times (0.12 + 0.15 + 0.08) = 1.35 < 3$$

| 字符       | 概率   | 编码 |
|----------|------|----|
| <i>a</i> | 0.12 | 01 |
| <i>b</i> | 0.40 | 0  |
| <i>c</i> | 0.15 | 00 |
| <i>d</i> | 0.08 | 10 |
| <i>e</i> | 0.25 | 1  |



# 变长编码



| 字符       | 概率   | 编码 |
|----------|------|----|
| <i>a</i> | 0.12 | 01 |
| <i>b</i> | 0.40 | 0  |
| <i>c</i> | 0.15 | 00 |
| <i>d</i> | 0.08 | 10 |
| <i>e</i> | 0.25 | 1  |

某些字符的码字是另一些字符的前缀，无法唯一恢复原始文本



# 前缀编码

- 前缀编码：编码后任一字符的编码都不是其它字符编码的前缀
- 平均码长： $\sum_{i=1}^5 l_i p_i = 2 \times (0.40 + 0.15 + 0.25) + 3 \times (0.12 + 0.18) = 2.2$
- 是否存在码长最短的前缀编码方案？

| 字符       | 概率   | 编码  |
|----------|------|-----|
| <i>a</i> | 0.12 | 000 |
| <i>b</i> | 0.40 | 11  |
| <i>c</i> | 0.15 | 01  |
| <i>d</i> | 0.08 | 001 |
| <i>e</i> | 0.25 | 10  |

01 译码  *c*      0101 译码  *cc*



# 变长编码的例子：Morse电码

- 用点(·)和划(—) 编码，  
主要用于电报系统传输  
英文信息



- 编码的长度不同，称为  
变长编码

## International Morse Code

1. The length of a dot is one unit.
2. A dash is three units.
3. The space between parts of the same letter is one unit.
4. The space between letters is three units.
5. The space between words is seven units.

A ● —  
B — ● ● ●  
C — ● — ●  
D — ● ●  
E ●  
F ● ● — ●  
G — — ●  
H ● ● ● ●  
I ● ●  
J ● — — —  
K — ● —  
L ● — ● ●  
M — —  
N — ●  
O — — —  
P ● — — ●  
Q — — ● —  
R ● — ●  
S ● ● ●  
T —

U ● ● —  
V ● ● ● —  
W ● — —  
X — ● ● —  
Y — ● — —  
Z — — ● ●

1 ● — — — —  
2 ● ● — — —  
3 ● ● ● — —  
4 ● ● ● ● —  
5 ● ● ● ● ●  
6 — ● ● ● ●  
7 — — ● ● ●  
8 — — — ● ●  
9 — — — — ●  
0 — — — — —



# 二进制编码的二叉树表示

- 任何一个编码方案都可以表示成一个二叉树 (0/1)
- 由根沿着二叉树的路径下行
  - 通向左孩子的分支标记为0
  - 通向右孩子的分支标记为1
- 叶子结点表示字符
- 到达叶子结点的路径表示编码方案

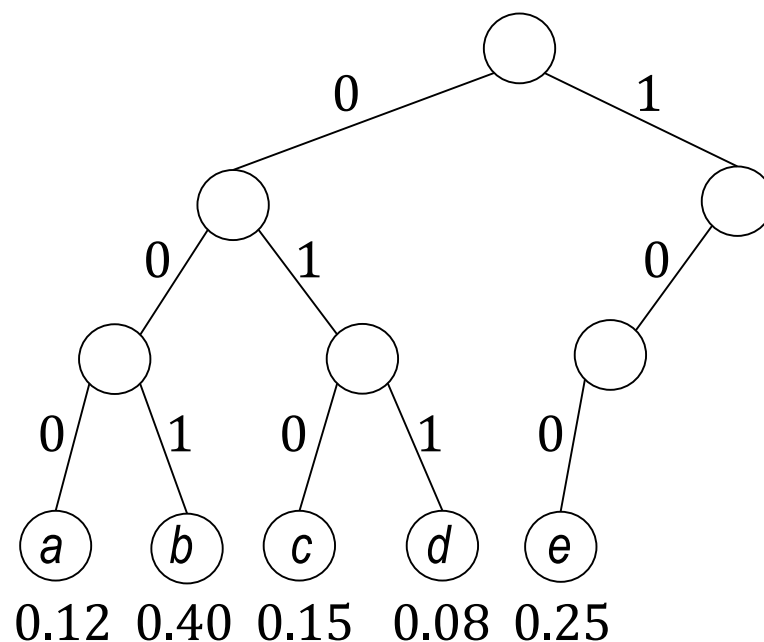
能把编码问题描述为一颗二叉树的问题，是最关键的！



# 二进制编码的叉树表示

前面定长编码的例子：

| 字符       | 概率   | 码字  |
|----------|------|-----|
| <i>a</i> | 0.12 | 000 |
| <i>b</i> | 0.40 | 001 |
| <i>c</i> | 0.15 | 010 |
| <i>d</i> | 0.08 | 011 |
| <i>e</i> | 0.25 | 100 |

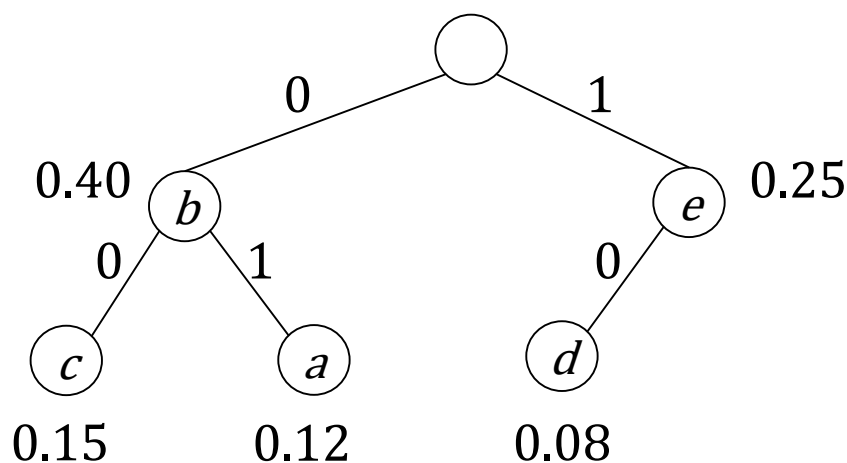






# 二进制编码的叉树表示

前面变长编码的例子：



| 字符       | 概率   | 编码 |
|----------|------|----|
| <i>a</i> | 0.12 | 01 |
| <i>b</i> | 0.40 | 0  |
| <i>c</i> | 0.15 | 00 |
| <i>d</i> | 0.08 | 10 |
| <i>e</i> | 0.25 | 1  |

用叶子结点表示字符，保证是前缀编码！

# 二进制编码的二叉树表示 - 随想

二叉树是二叉结点的

也是叶子结点的

但归根结底是叶子结点的

叶子结点们

迎着早晨八九点钟的太阳，

(最优编码的) 希望寄托在他们身上





# 霍夫曼编码

- 霍夫曼编码是依据字符出现概率来构造平均编码长度最短编码的方案，有时也称为最佳编码，一般就叫作霍夫曼编码
- 霍夫曼编码满足前缀编码的性质，因此可以唯一译码



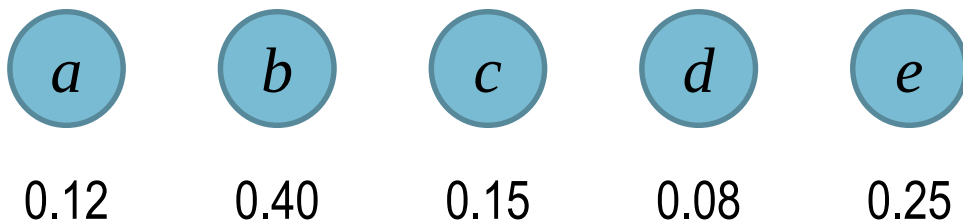
# 霍夫曼编码

- 霍夫曼编码的过程就是构造霍夫曼树的过程
- 字符出现的概率就是霍夫曼树中叶子结点的路径权重

| 字符       | 概率   | 编码 |
|----------|------|----|
| <i>a</i> | 0.12 |    |
| <i>b</i> | 0.40 |    |
| <i>c</i> | 0.15 |    |
| <i>d</i> | 0.08 |    |
| <i>e</i> | 0.25 |    |

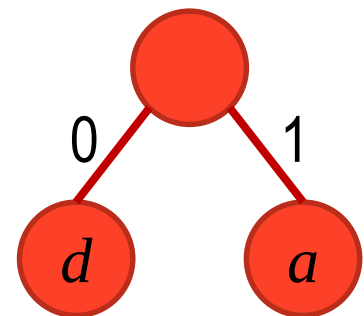
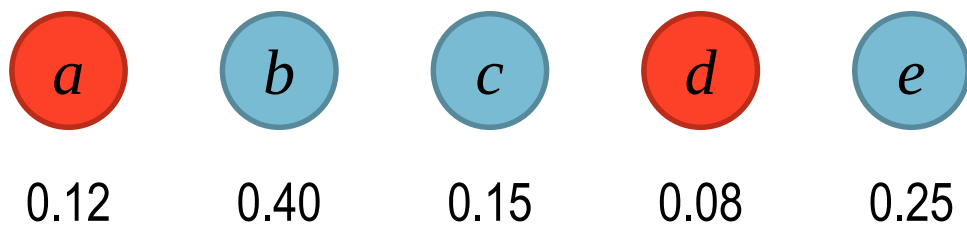


# 霍夫曼编码 – 举例



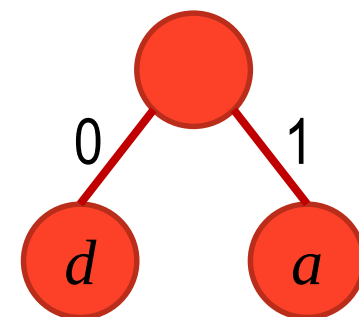
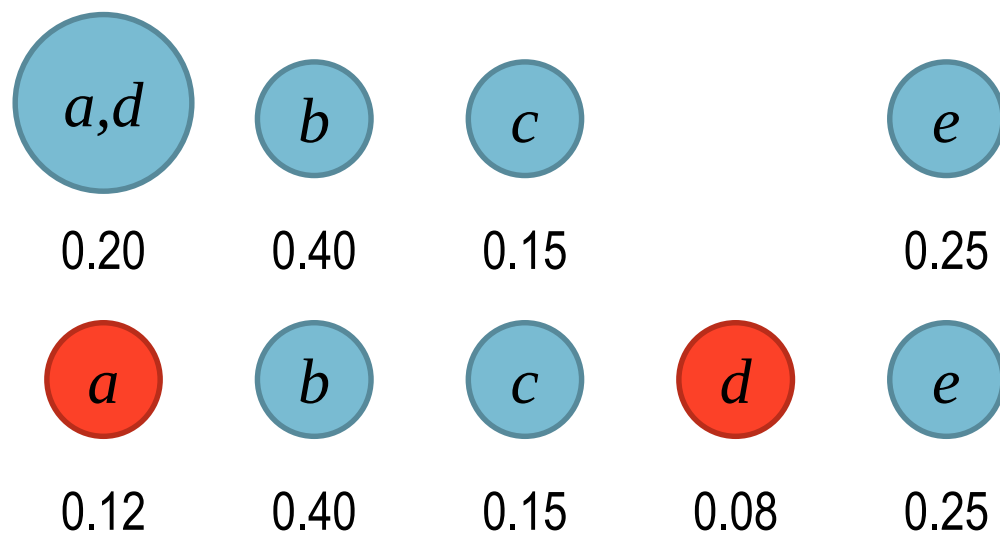


# 霍夫曼编码 – 举例



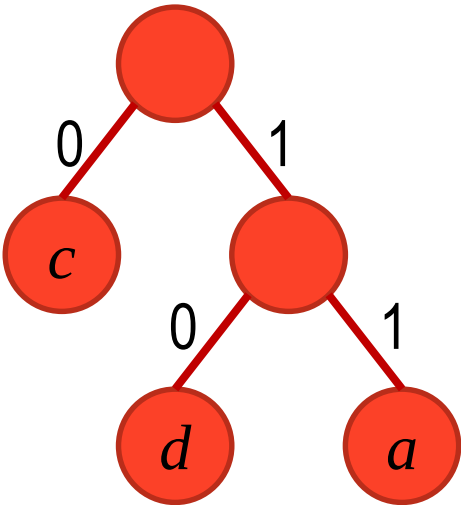
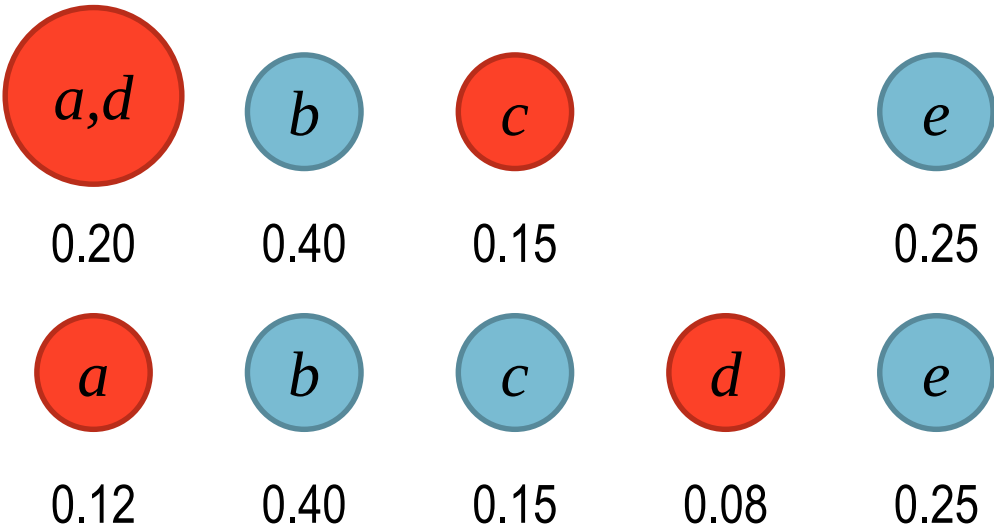


# 霍夫曼编码 – 举例



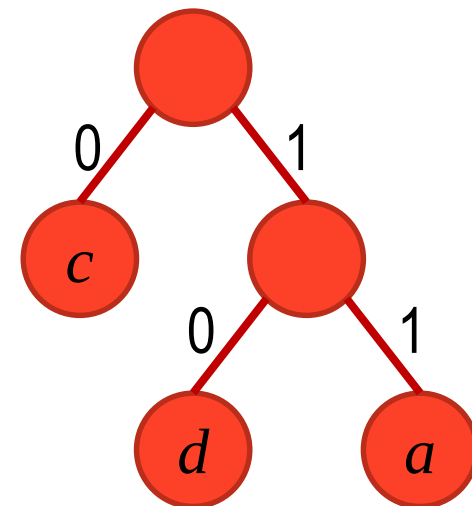
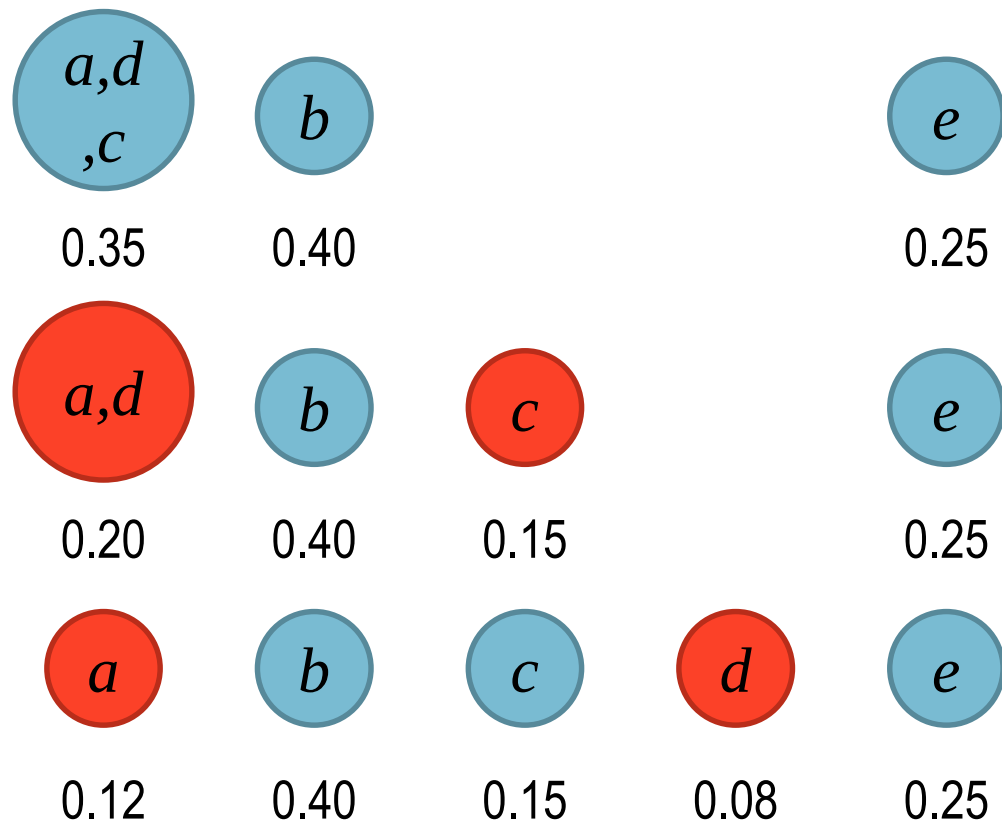


# 霍夫曼编码 – 举例

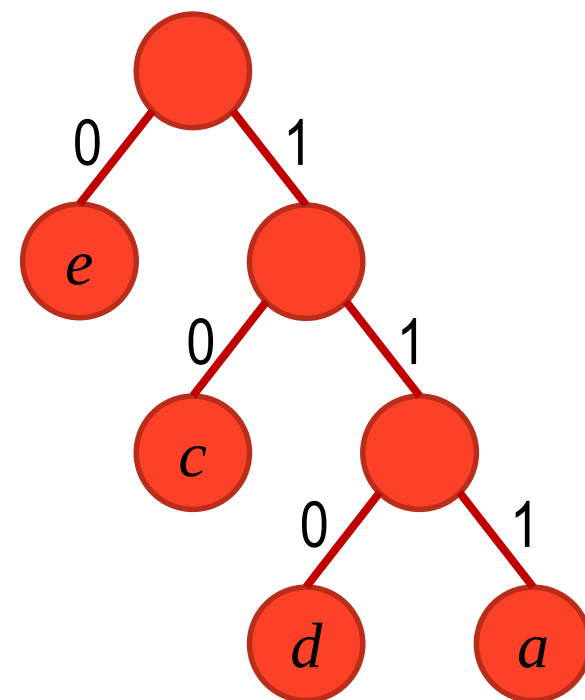
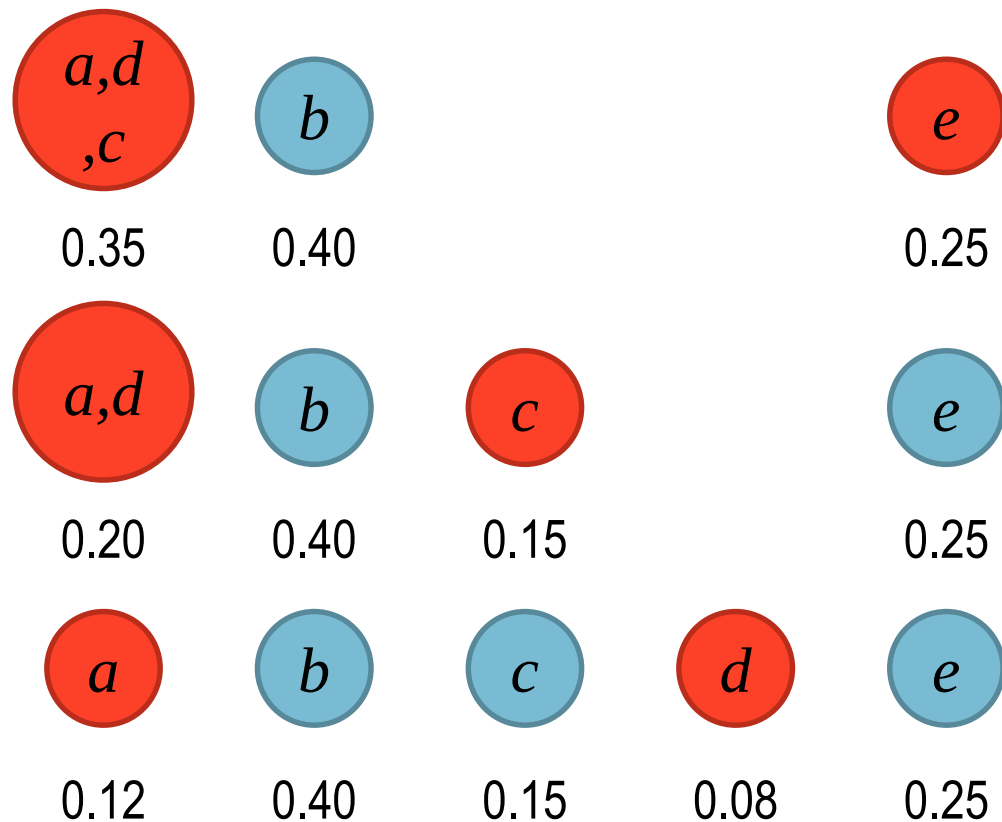




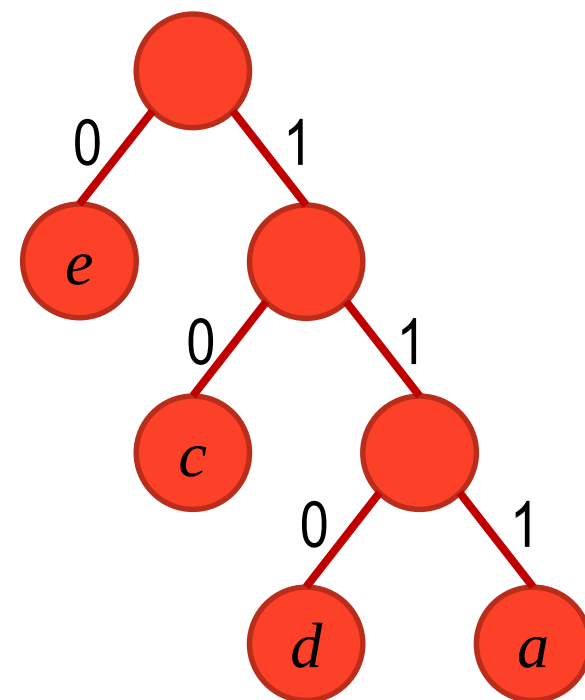
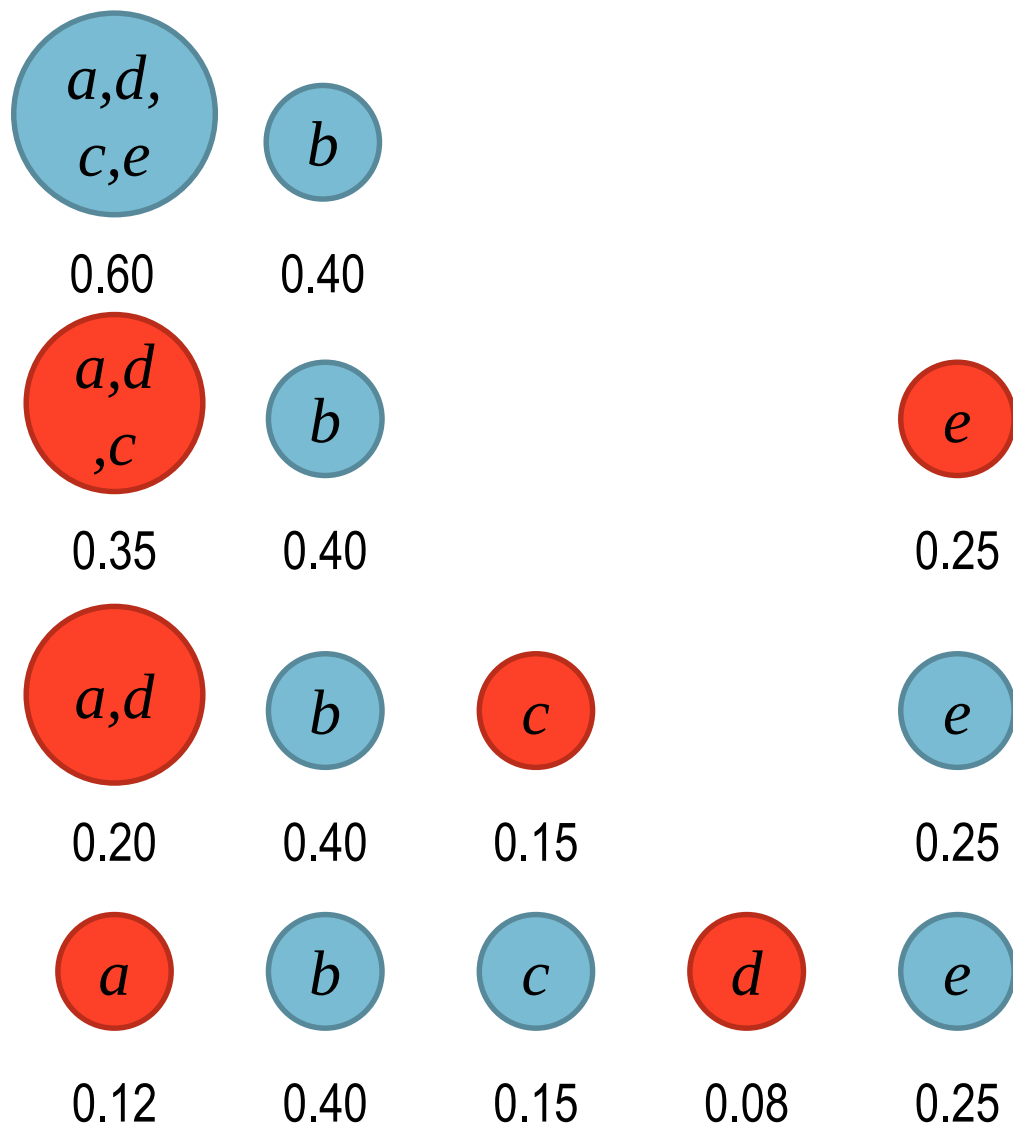
# 霍夫曼编码 – 举例



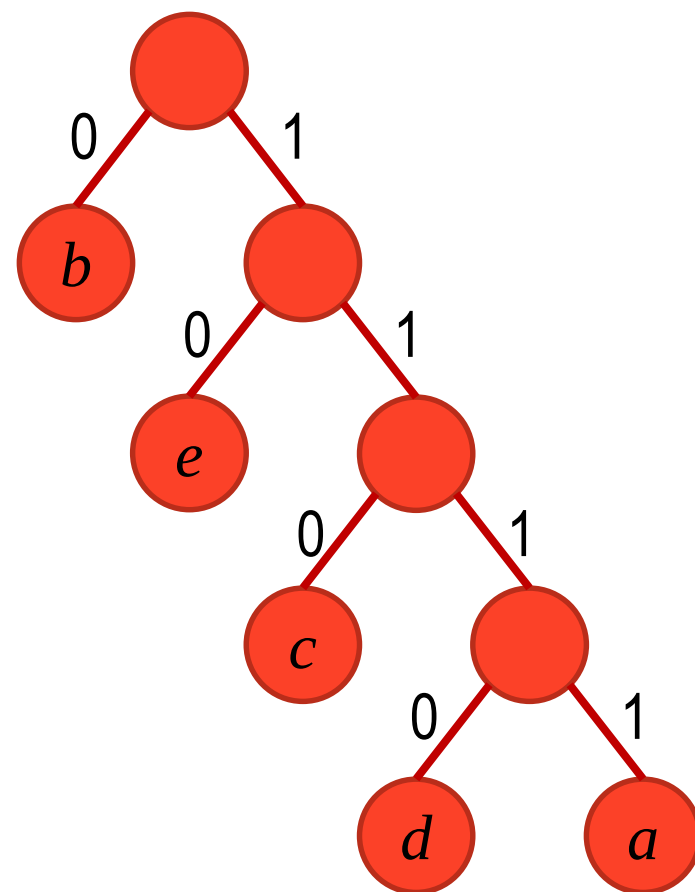
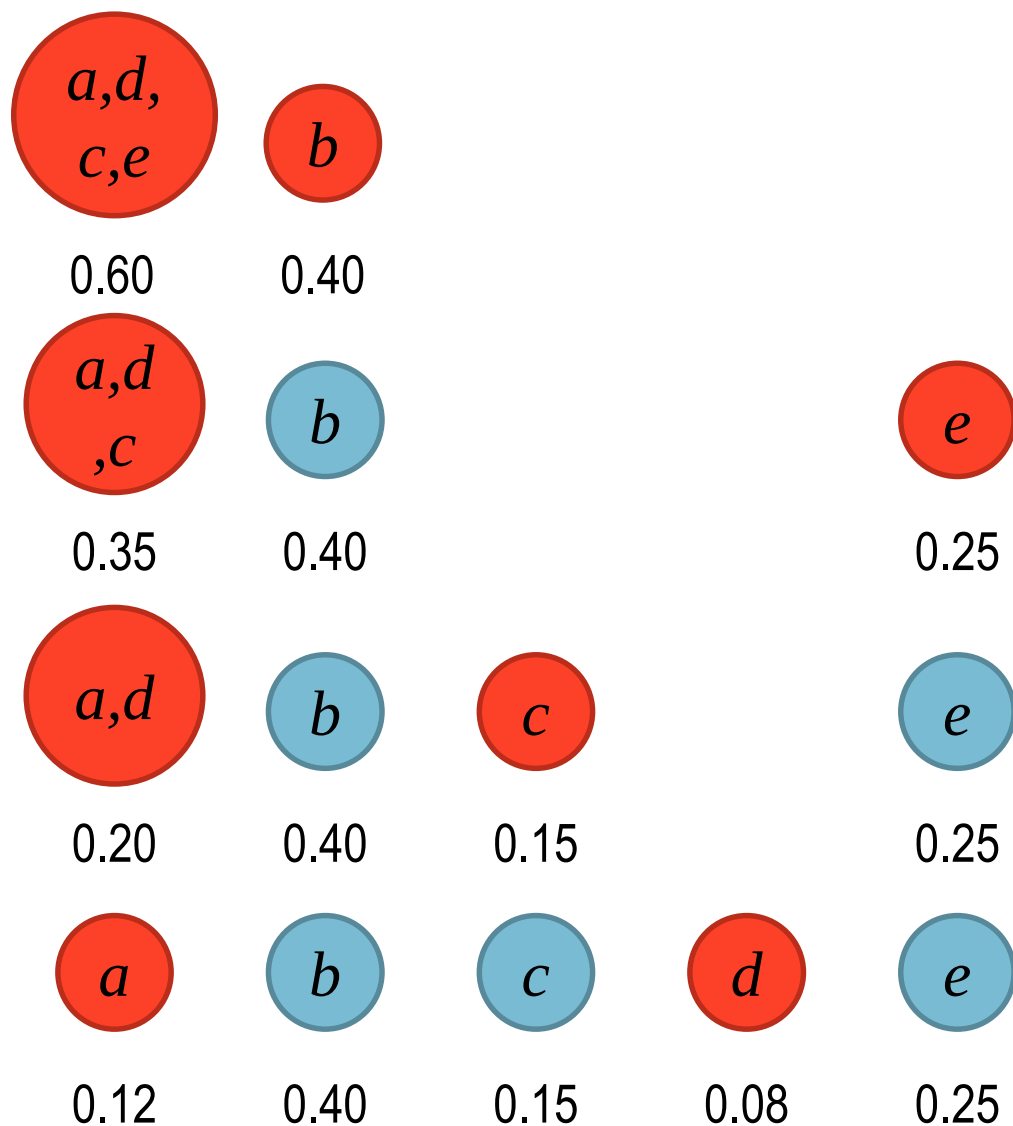
# 霍夫曼编码 – 举例



# 霍夫曼编码 – 举例



# 霍夫曼编码 – 举例



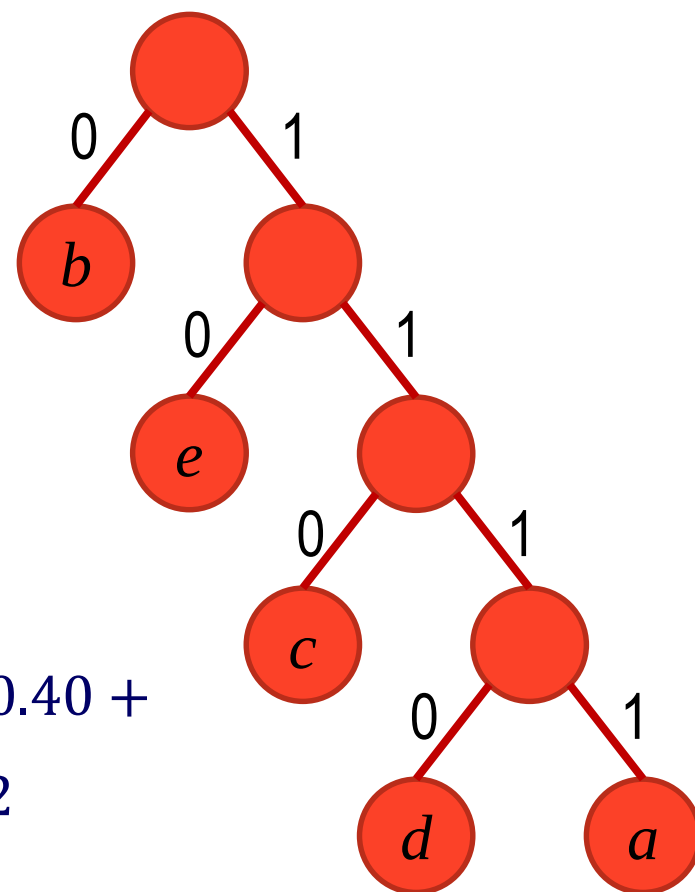


# 霍夫曼编码 – 举例

| 字符       | 概率   | 编码   |
|----------|------|------|
| <i>a</i> | 0.12 | 1111 |
| <i>b</i> | 0.40 | 0    |
| <i>c</i> | 0.15 | 110  |
| <i>d</i> | 0.08 | 1110 |
| <i>e</i> | 0.25 | 10   |

平均编码长度： $\sum_{i=1}^5 l_i p_i = 4 \times 0.12 + 1 \times 0.40 + 3 \times 0.15 + 4 \times 0.08 + 2 \times 0.25 = 2.15 < 2.2$

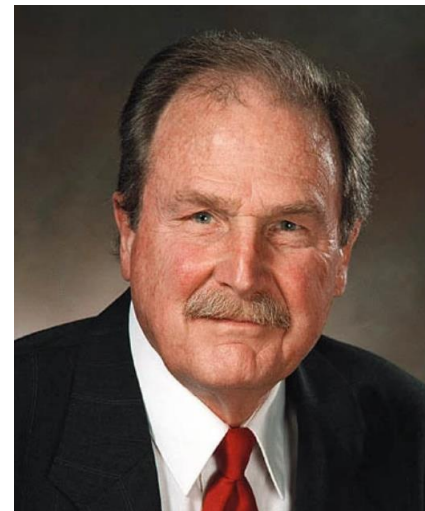
优于前面的定长和变长编码方案





# 关于霍夫曼 (D. A. Huffman)

- 17岁在俄亥俄州立大学毕业
- 1950年，霍夫曼在MIT攻读博士学位。信息理论与编码课上，Robert Fano教授让学生们自己决定是参加期末考试还是做一个大作业。霍夫曼选择了后者，诞生了霍夫曼编码



As a scientist and a teacher, I am terribly tenacious. If I sense that I have not yet arrived at the simplest possible way of looking at a problem, then I am dissatisfied until I have. That, to me, is the essence of being a scientist



# 提纲

- 二叉搜索树
- 霍夫曼树与霍夫曼编码
- 堆



# 堆 (Heap)

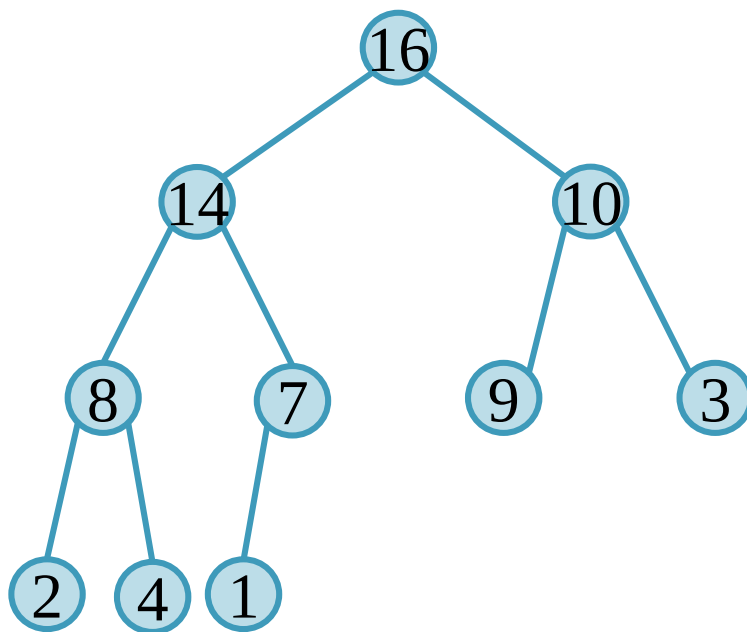
- 堆 (Heap) 是一种满足特定性质的完全二叉树
- 最大堆
  - 堆中任一结点的值都小于其父结点的值，即对于所有结点  $i > 0$ ，有  $A[\text{parent}(i)] > A[i]$
  - 堆顶结点的值最大
- 最小堆
  - 堆中任一结点的值都大于其父结点的值，即对于所有结点  $i > 0$ ，有  $A[\text{parent}(i)] < A[i]$
  - 堆顶结点的值最小





# 最大堆的例子

- 堆中任一结点的值都小于其父结点的值，即对于所有结点  $i > 0$ ，有  $A[\text{parent}(i)] > A[i]$
- 堆顶结点的值最大





# 堆 (Heap)

- 堆可以顺序存储在一维数组 $A$ 中（适于完全二叉树）
  - $A[0]$ 存储根结点, 第 $i$ 个结点为 $A[i]$ ,  $A[i]$ 的父亲结点为 $A\left[\frac{i-1}{2}\right]$
  - $A[i]$ 的左孩子结点为 $A[2i + 1]$ ,  $A[i]$ 的右孩子结点为 $A[2i + 2]$
- 堆的高度为堆顶结点的高度, 有 $n$ 个结点的堆的高度为 $\lceil \log_2(n + 1) \rceil$ 
  - 树中结点的高度为从该结点到叶子结点的最长路径
  - 树的高度定义为根的高度（与上一讲树高度的描述不同, 本质相同）



# 堆的抽象数据类型

ADT Heap{

数据对象:  $D = \{a_i \mid a_i \in ElemSet, i = 0, \dots, n, n \geq 0\}$

数据关系:  $R_1 = \{a_i \geq a_{2i+1} \text{ 且 } a_i \geq a_{2i+2} \mid a_i, a_{2i+1}, a_{2i+2} \in D, i = 0, 1, \dots, n\}$

基本操作:

InitHeap(&H);

操作结果: 构造空堆H

DestroyHeap(&H);

初始条件: 堆H已存在

操作结果: 销毁堆H

ClearHeap(&H)

初始条件: 堆H已存在

操作结果: 将堆H清为空堆



# 堆的抽象数据类型

IsEmpty(H)

初始条件：堆H已存在

操作结果：若堆H为空，则返回TRUE，否则返回FALSE

IsFull(H)

初始条件：堆H已存在

操作结果：若堆H已满，则返回TRUE，否则返回FALSE

GetLength(H)

初始条件：堆H已存在

操作结果：返回堆H中的元素个数，即堆的规模

Insert(&H, e)

初始条件：堆H已存在

操作结果：向堆中插入结点e, 并仍然保持堆性质



# 堆的抽象数据类型

Remove(&H, i)

初始条件：堆H已存在

操作结果：从堆中删除堆顶结点,并仍然保持堆性质

FixUp(&H, i)

初始条件：堆H已存在

操作结果：从堆的第i个结点,自底向上堆化

FixDown(&H, i)

初始条件：堆H已存在

操作结果：从堆的第i个结点,自顶向下堆化

}ADT Heap



# 堆的操作

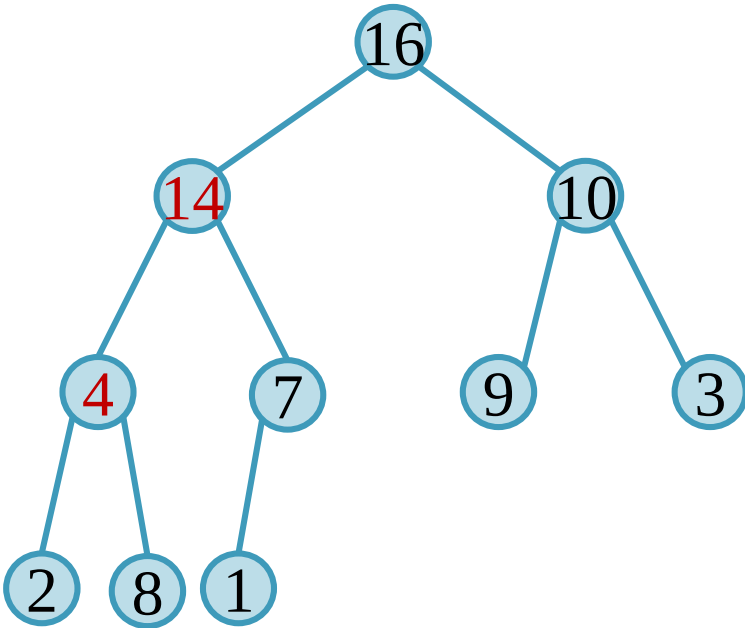
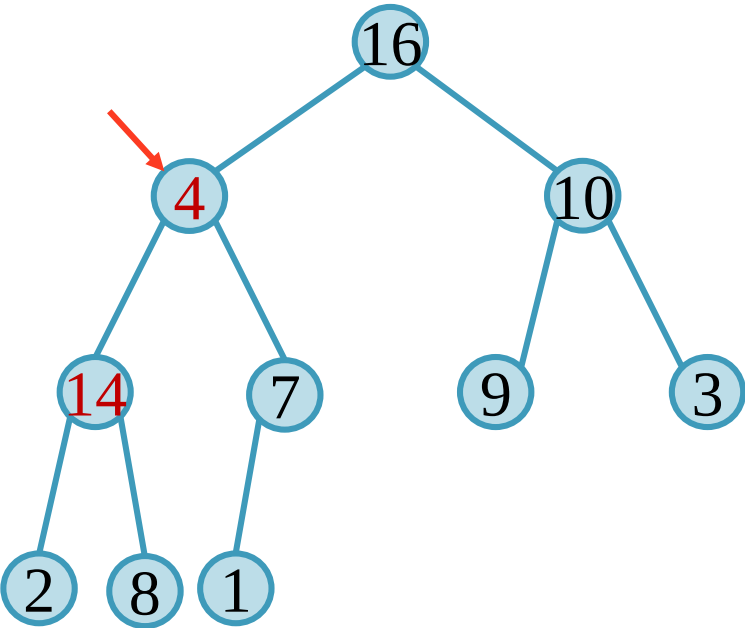
- 对于堆的操作，如插入、删除、或修改结点的值都会破坏堆的性质，因此对堆进行修复以维护堆性质是堆最重要的操作
- 对于最大堆，如果某个结点的值小于其某个子结点 (向下看发现问题)，可以采用自顶向下堆化(Heapify-down)的算法进行修复
  - ① 计算当前结点两个子结点中较大者
  - ② 比较，如果满足堆的性质则终止，否则与其父结点交换
  - ③ 取交换后的结点（第①步较大子结点的位置）为当前结点，并转向第①步
  - ④ 执行上述过程直至每个结点都满足最大堆的性质



# 堆的操作 – 自顶向下堆化

|    |   |    |    |   |   |   |   |   |   |
|----|---|----|----|---|---|---|---|---|---|
| 16 | 4 | 10 | 14 | 7 | 9 | 3 | 2 | 8 | 1 |
|----|---|----|----|---|---|---|---|---|---|

|    |    |    |   |   |   |   |   |   |   |
|----|----|----|---|---|---|---|---|---|---|
| 16 | 14 | 10 | 4 | 7 | 9 | 3 | 2 | 8 | 1 |
|----|----|----|---|---|---|---|---|---|---|

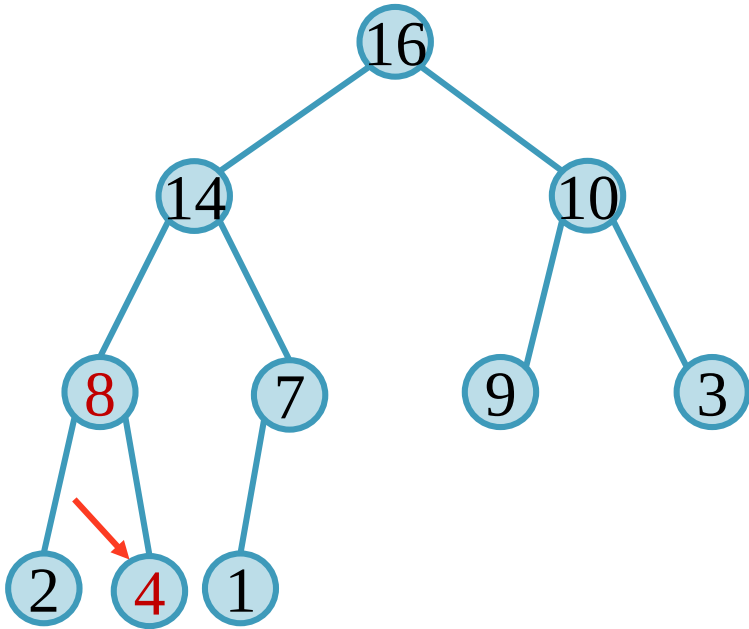
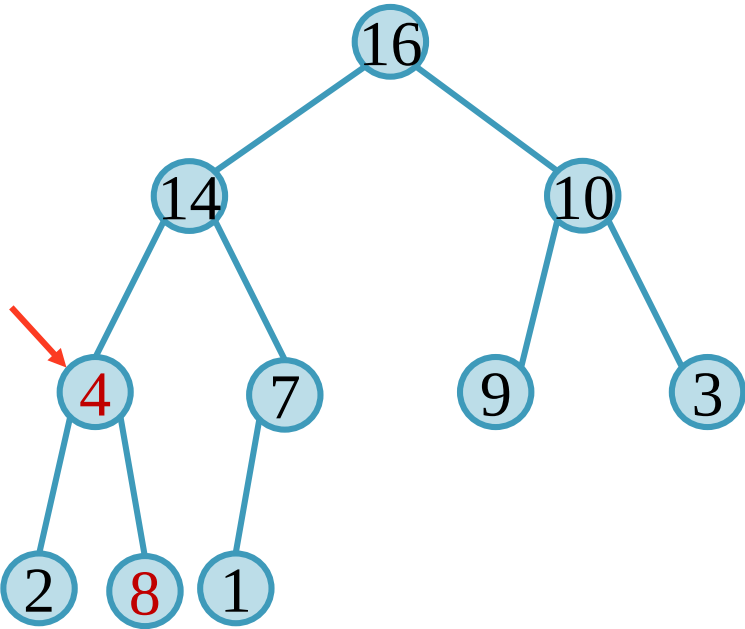




# 堆的操作 – 自顶向下堆化

|    |    |    |   |   |   |   |   |   |   |
|----|----|----|---|---|---|---|---|---|---|
| 16 | 14 | 10 | 4 | 7 | 9 | 3 | 2 | 8 | 1 |
|----|----|----|---|---|---|---|---|---|---|

|    |    |    |   |   |   |   |   |   |   |
|----|----|----|---|---|---|---|---|---|---|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1 |
|----|----|----|---|---|---|---|---|---|---|







# 堆的操作 – 自顶向下堆化

```
void CMaxHeap::FixDown(int k){
```

```
    int i;
```

```
    i = 2*k+1;
```

//左孩子结点位置

```
    while(i < heapSize)    {
```

```
        if(i < heapSize-1 && heap[i] < heap[i+1])
```

//取孩子结点中较大者

```
            i++;
```

```
        if(heap[k] > heap[i]) break;
```

//满足堆性质则终止

```
        exch(heap[k],heap[i]);
```

//交换双亲和孩子结点

```
        k = i;
```

//新的当前结点位置

```
        i = 2*k+1;
```

//新的左孩子结点位置

```
    }
```

```
}
```



# 堆的操作 – 自顶向下堆化

- 最坏情况计算复杂度为 $O(h)$ ,  $h$ 为堆的高度
- 本地操作, 基本不需要辅助空间



# 堆的操作 – 自底向上堆化

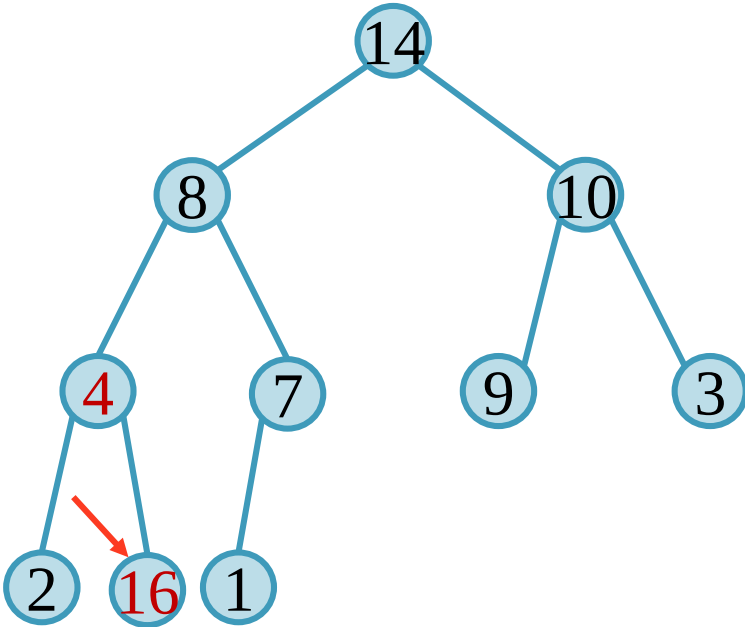
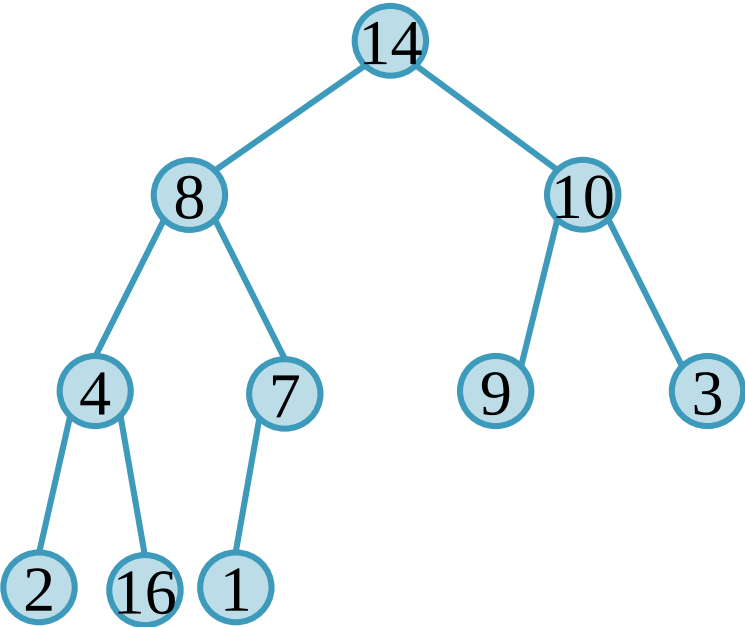
- 对最大堆，如果某个结点的值大于其父结点 (向上看发现问题)，可以采用**自底向上堆化**(Heapify-up)的算法进行修复
- 执行过程
  - ① 找到当前结点的父结点
  - ② 比较，如果满足堆的性质则终止，否则与其父结点交换
  - ③ 取交换后的结点（第②步中父结点的位置）为当前结点，并转向第①步
  - ④ 执行上述过程直至每个结点都满足最大堆的性质
- 最坏情况计算复杂度为 $O(h)$ ， $h$ 为堆的高度
- 本地操作，基本不需要辅助空间



# 堆的操作 – 自底向上堆化

|    |   |    |   |   |   |   |   |    |   |
|----|---|----|---|---|---|---|---|----|---|
| 14 | 8 | 10 | 4 | 7 | 9 | 3 | 2 | 16 | 1 |
|----|---|----|---|---|---|---|---|----|---|

|    |   |    |   |   |   |   |   |    |   |
|----|---|----|---|---|---|---|---|----|---|
| 14 | 8 | 10 | 4 | 7 | 9 | 3 | 2 | 16 | 1 |
|----|---|----|---|---|---|---|---|----|---|

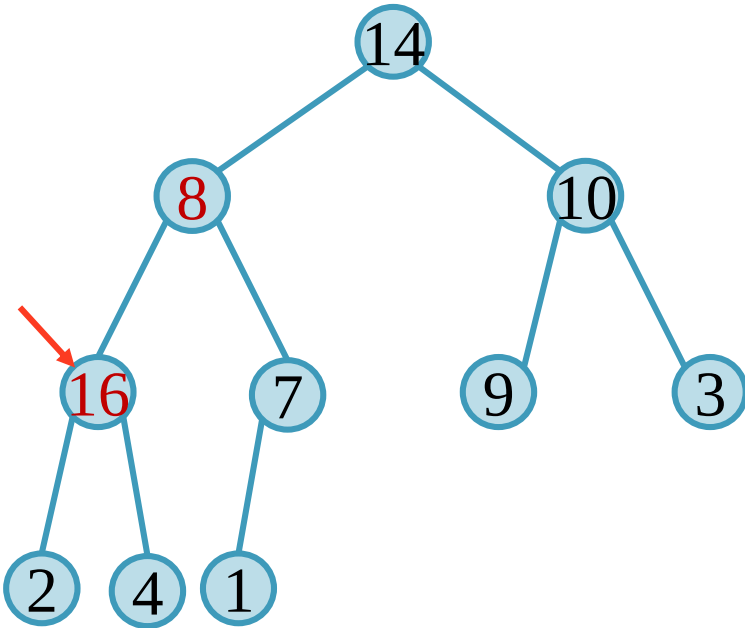
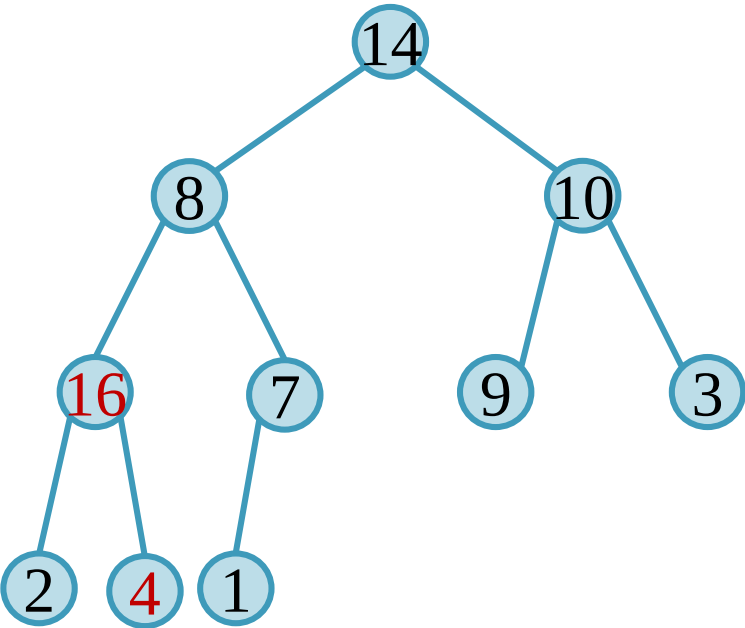




# 堆的操作 – 自底向上堆化

|    |   |    |    |   |   |   |   |   |   |
|----|---|----|----|---|---|---|---|---|---|
| 14 | 8 | 10 | 16 | 7 | 9 | 3 | 2 | 4 | 1 |
|----|---|----|----|---|---|---|---|---|---|

|    |   |    |    |   |   |   |   |   |   |
|----|---|----|----|---|---|---|---|---|---|
| 14 | 8 | 10 | 16 | 7 | 9 | 3 | 2 | 4 | 1 |
|----|---|----|----|---|---|---|---|---|---|

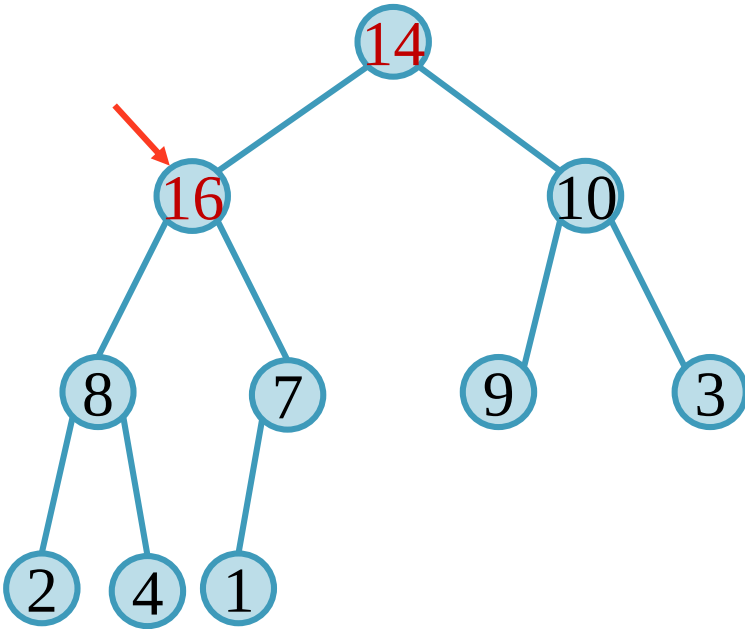
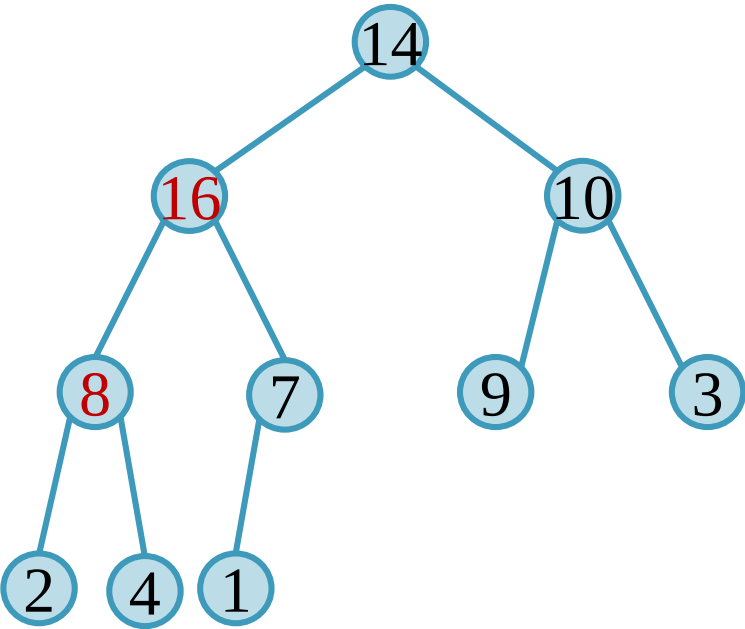




# 堆的操作 – 自底向上堆化

|    |    |    |   |   |   |   |   |   |   |
|----|----|----|---|---|---|---|---|---|---|
| 14 | 16 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1 |
|----|----|----|---|---|---|---|---|---|---|

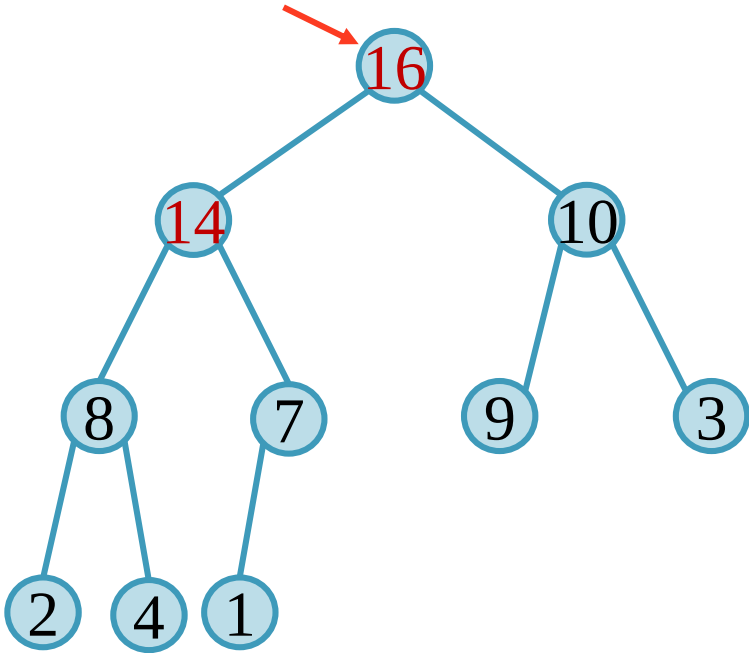
|    |    |    |   |   |   |   |   |   |   |
|----|----|----|---|---|---|---|---|---|---|
| 14 | 16 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1 |
|----|----|----|---|---|---|---|---|---|---|





# 堆的操作 – 自底向上堆化

|    |    |    |   |   |   |   |   |   |   |
|----|----|----|---|---|---|---|---|---|---|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1 |
|----|----|----|---|---|---|---|---|---|---|





# 堆的操作 – 自底向上堆化

```
void CMaxHeap::FixUp(int k) {  
    int i;  
    i = (k-1)/2;  
    while(k > 0 && heap[i] < heap[k]) {  
        exch(heap[k],heap[i]);  
        k = i;  
        i = (k-1)/2;  
    }  
}
```

//取双亲结点位置

//不满足堆性质

//交换当前结点和双亲结点的值

//新的当前结点

//新的双亲结点

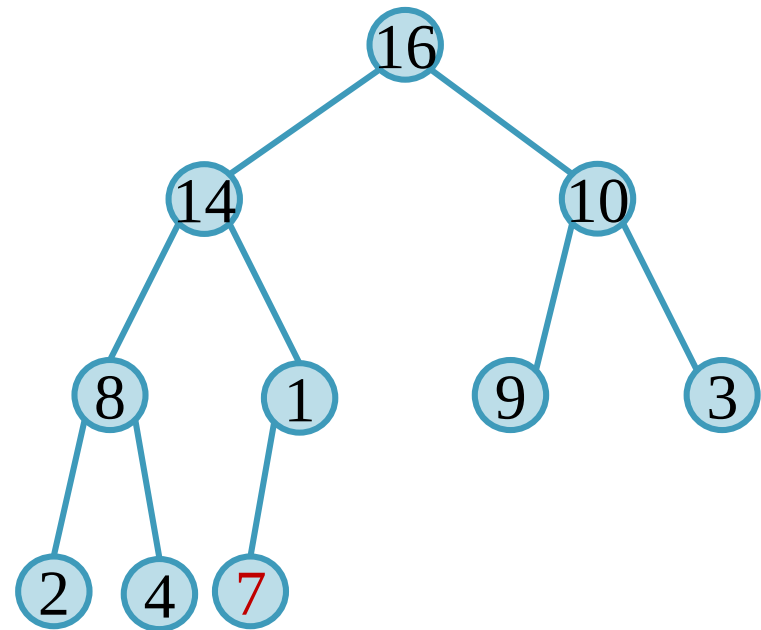




# 堆的操作 – 插入操作

- 将新结点加入堆尾
- 调用自底向上堆化算法调整堆

|    |    |    |   |   |   |   |   |   |   |
|----|----|----|---|---|---|---|---|---|---|
| 16 | 14 | 10 | 8 | 1 | 9 | 3 | 2 | 4 | 7 |
|----|----|----|---|---|---|---|---|---|---|

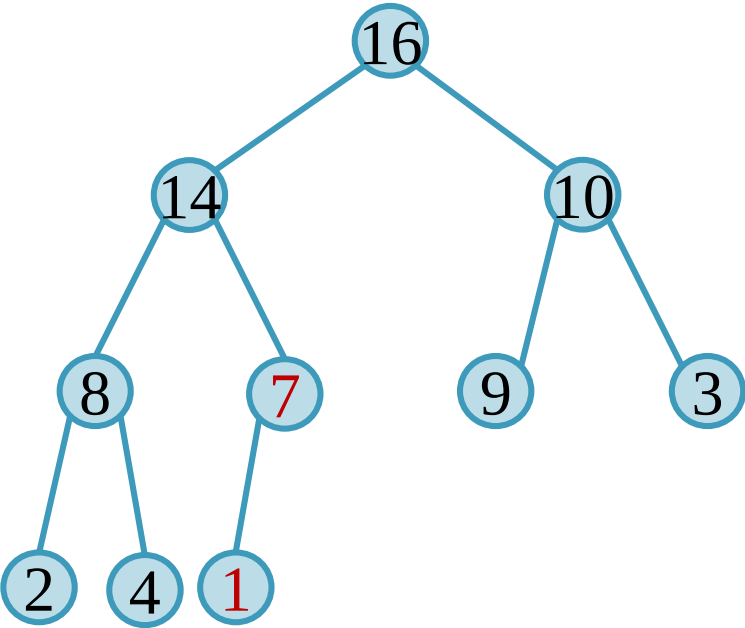


```
void CMaxHeap::Insert(ElemType e) {  
    heap[heapSize] = e;  
    heapSize ++;  
    FixUp(heapSize-1);  
}
```



# 堆的操作 – 插入操作

|    |    |    |   |   |   |   |   |   |   |
|----|----|----|---|---|---|---|---|---|---|
| 16 | 14 | 10 | 8 | 7 | 9 | 3 | 2 | 4 | 1 |
|----|----|----|---|---|---|---|---|---|---|





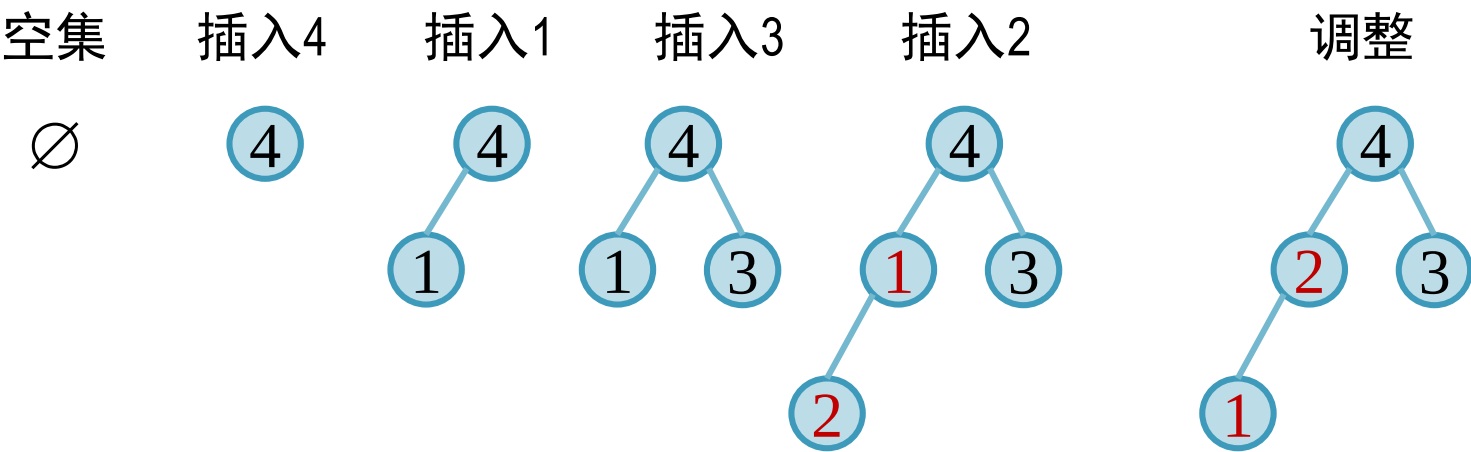
# 堆的操作 – 构造堆

- 自顶向下的堆构造
  - 不断向堆尾插入新结点，并调用自底向上堆化操作构造堆
  - 时间复杂度小于 $O(n \log n)$ ， $n$ 为结点数
- 自底向上的堆构造



# 堆的操作 – 自顶向下构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

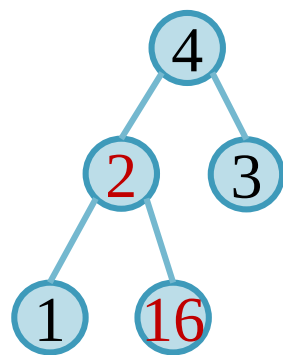




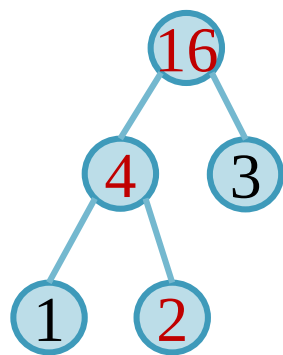
# 堆的操作 – 自顶向下构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

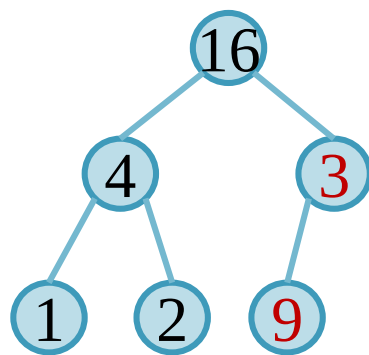
插入16



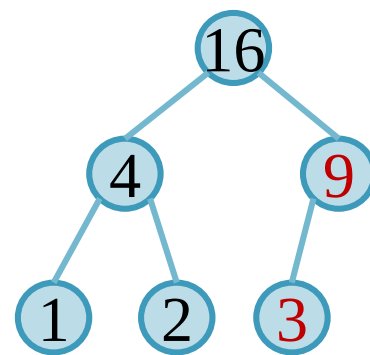
调整



插入9



调整

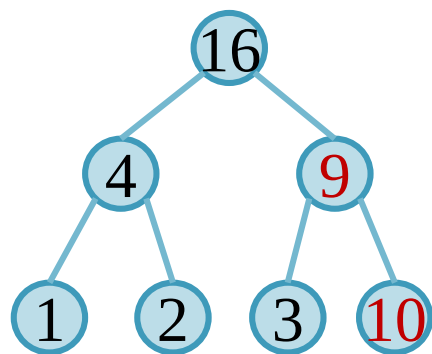




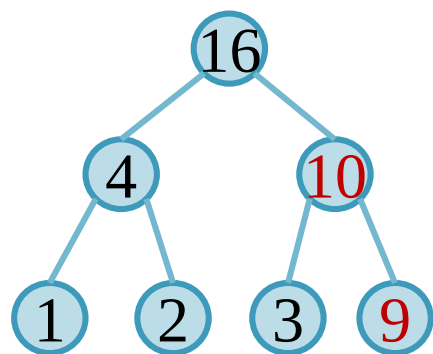
# 堆的操作 – 自顶向下构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

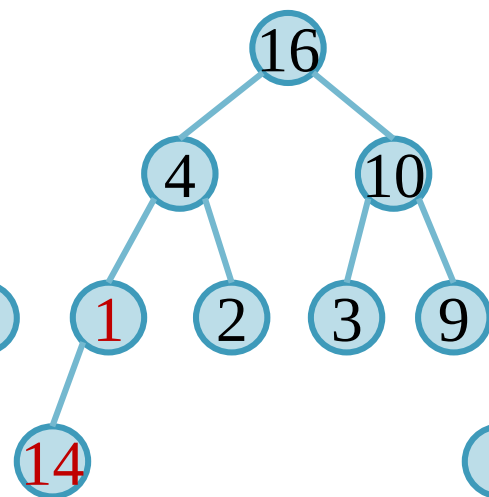
插入10



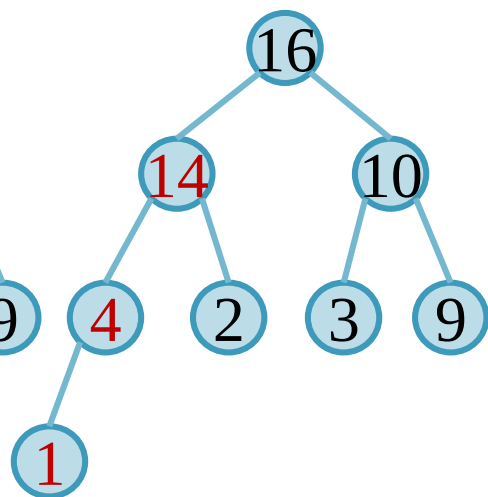
调整



插入14



调整

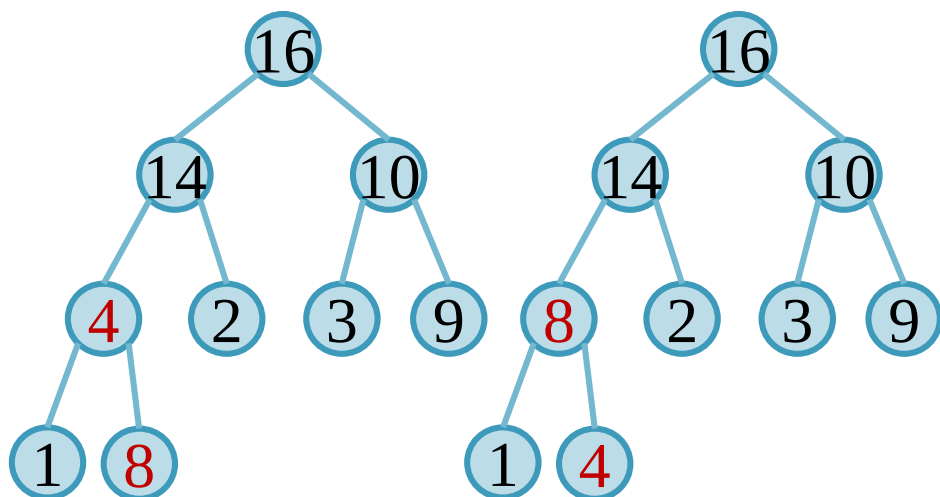




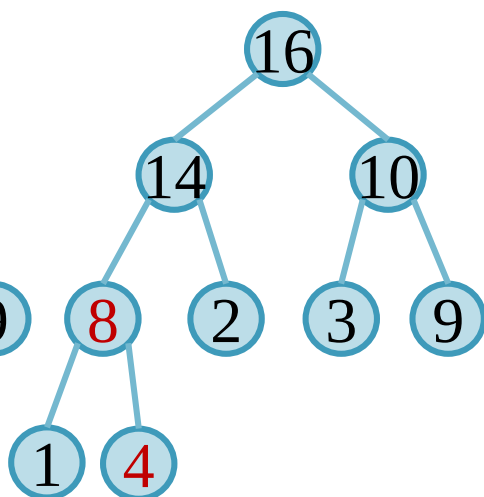
# 堆的操作 – 自顶向下构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

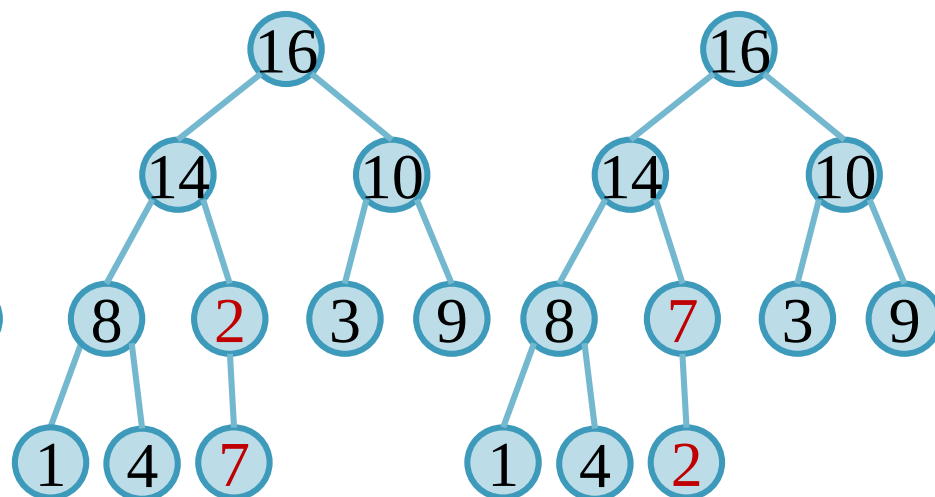
插入8



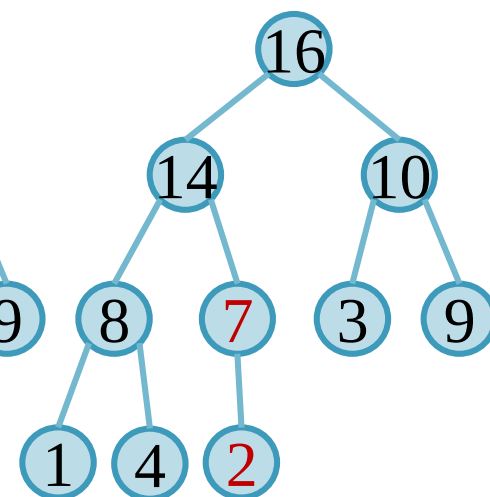
调整



插入7



调整





# 堆的操作 – 自底向上构造堆

- 将数组 $A$ 视为一棵尚未满足堆性质的完全二叉树
- 此时，长度为 $n$ 的二叉树 $A$ 中， $A \left[ \left\lfloor \frac{n}{2} \right\rfloor, \dots, n-1 \right]$ 是 $\left\lfloor \frac{n}{2} \right\rfloor$ 个堆
- 然后从 $\left\lfloor \frac{n}{2} \right\rfloor - 1$ 到0反向遍历数组(即二叉树的非叶子结点)，采用**自顶向下堆化操作**

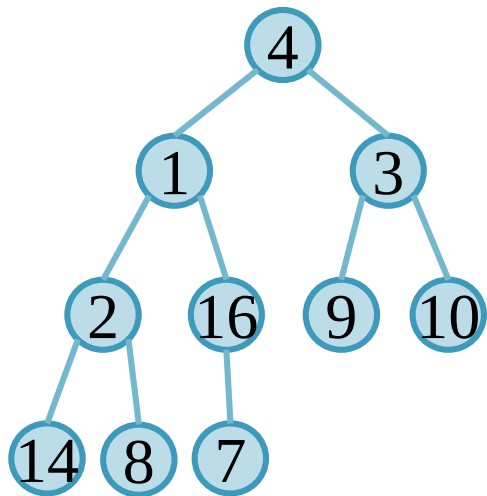




# 堆的操作 – 自底向上构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

初始

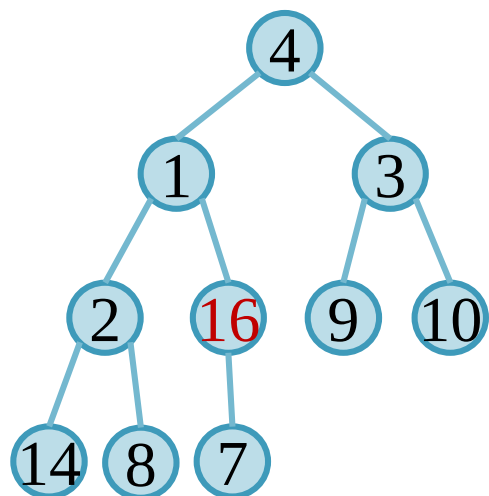




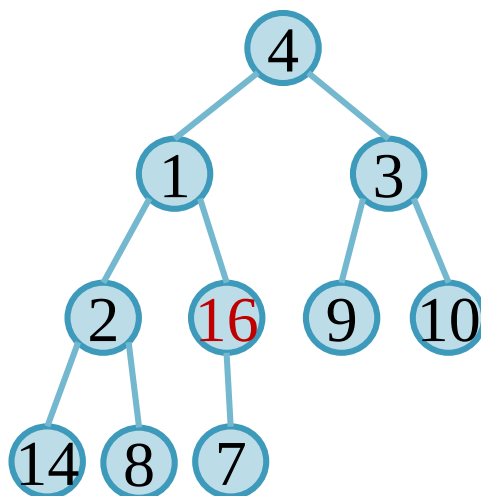
# 堆的操作 – 自底向上构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

元素16



调整

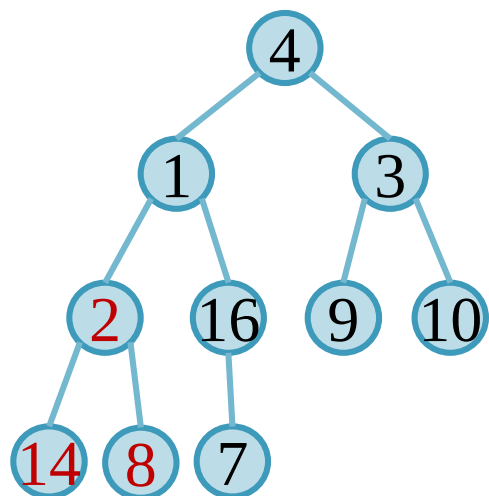




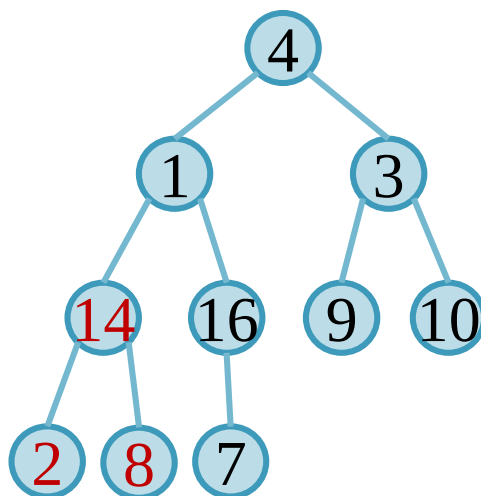
# 堆的操作 – 自底向上构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

元素 2



调整

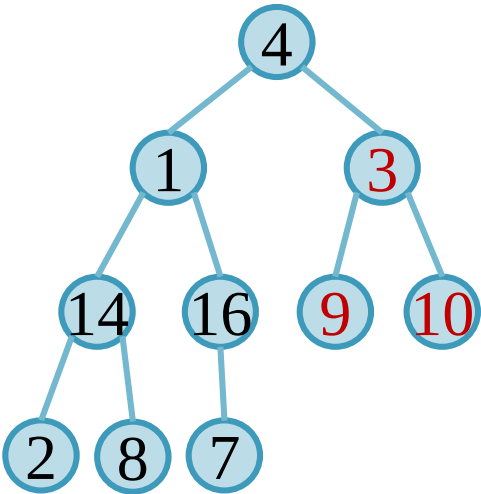




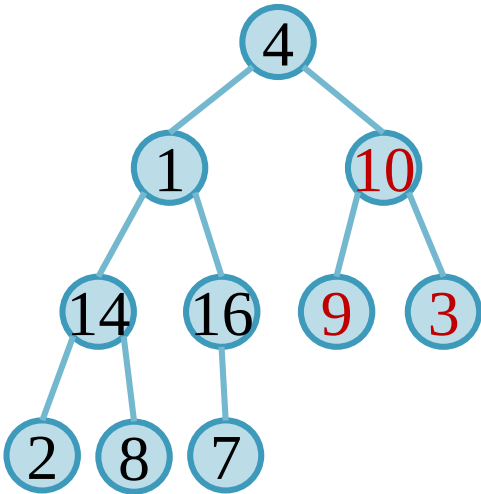
# 堆的操作 – 自底向上构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

元素 3



调整

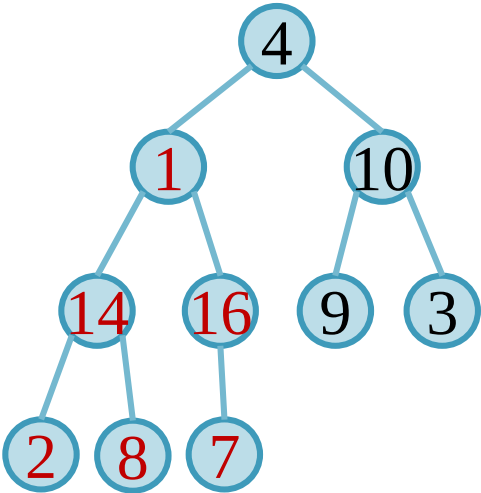




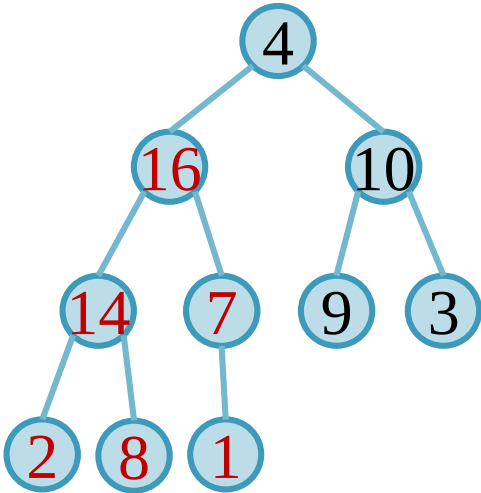
# 堆的操作 – 自底向上构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

元素 1



调整

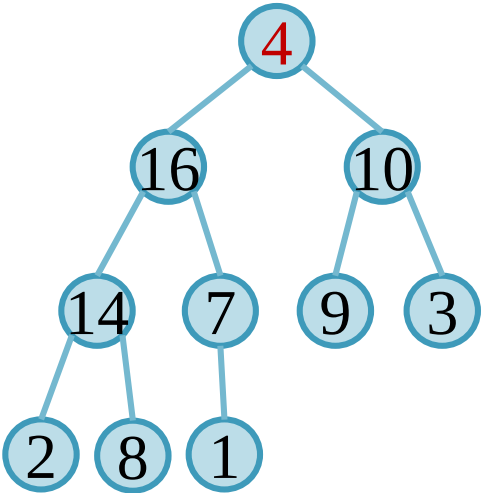




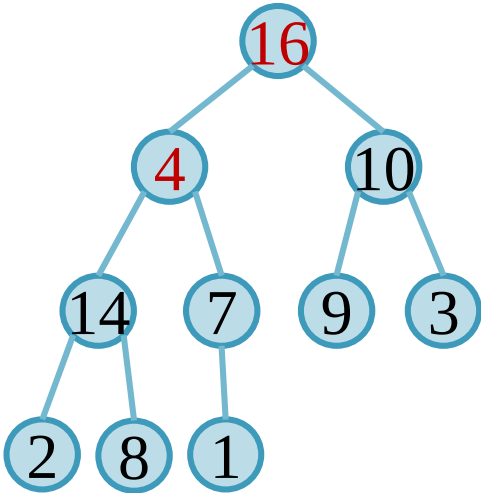
# 堆的操作 – 自底向上构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

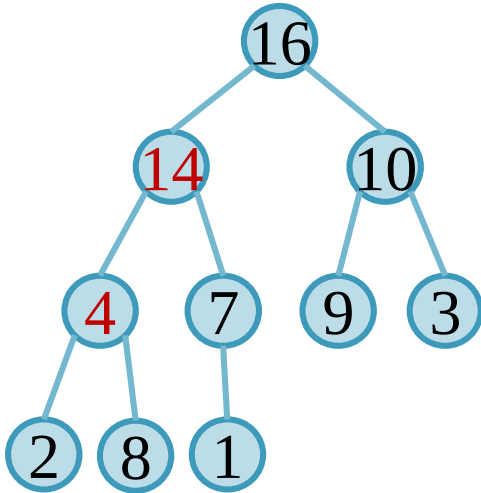
元素 4



调整



调整

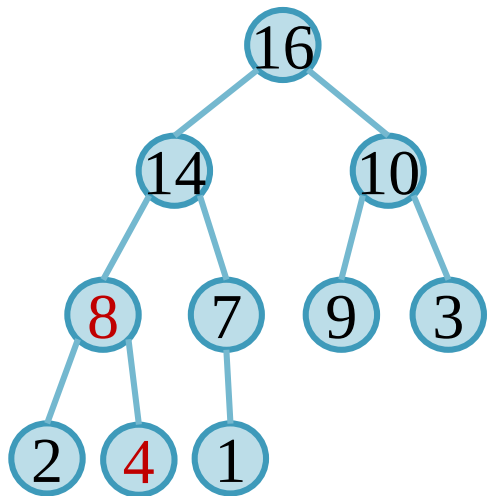




# 堆的操作 – 自底向上构造堆

$$A = \{4, 1, 3, 2, 16, 9, 10, 14, 8, 7\}$$

调整





# 堆的操作 – 自底向上构造堆

- 设堆的大小为 $n$ ，高度为 $h$ ，第 $k$ 层最多有 $2^{k-1}$ 个结点， $n \leq 2^h - 1$
- 对于第 $k$ 层的结点来说，自顶向下堆化操作最多是 $O(h - k)$
- 因此总的构造堆的时间复杂度为

$$\begin{aligned} T &= \sum_{k=1}^h 2^{k-1} O(h - k) = \sum_{k=0}^{h-1} 2^{h-k-1} O(k) = O\left(\sum_{k=0}^{h-1} 2^{h-k-1} k\right) \\ &= O\left(2^{h-1} \sum_{k=0}^{h-1} \frac{k}{2^k}\right) = O\left(2^{h-1} \times \left(2 - \frac{h+1}{2^{h-1}}\right)\right) = O(n) \end{aligned}$$

- 自底向上的堆构造是线性复杂度的

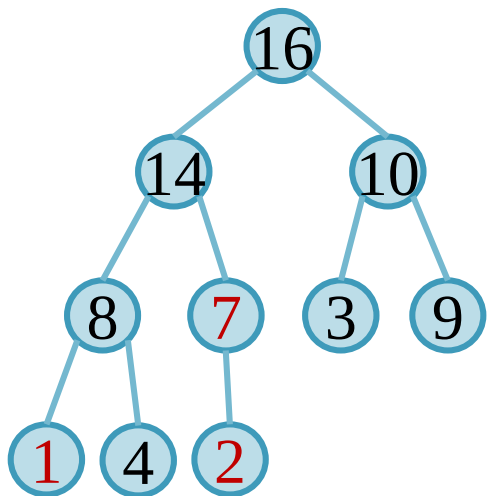




# 堆的操作 – 构造堆

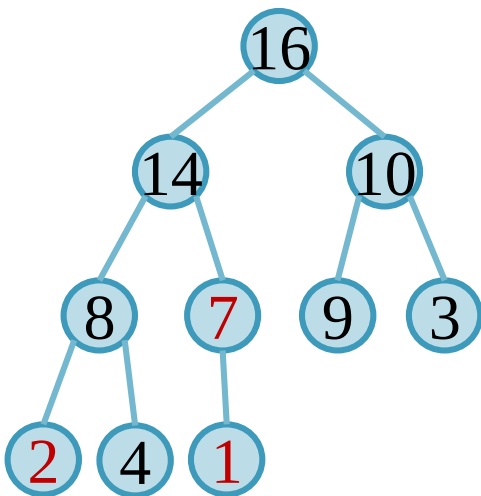
$$A = \{4,1,3,2,16,9,10,14,8,7\}$$

自顶向下



≠

自底向上

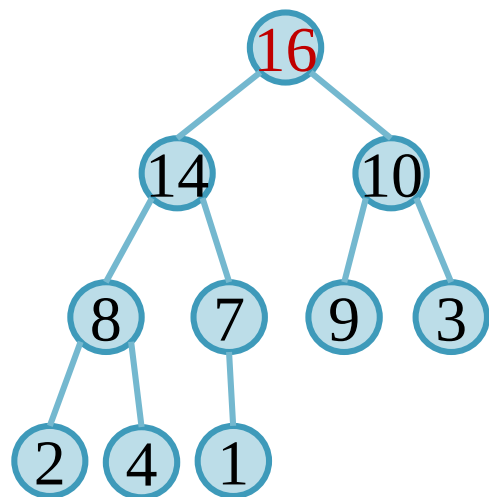




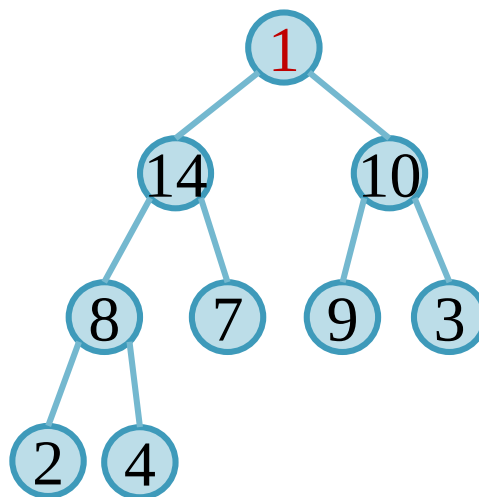
# 堆的操作 – 删除操作

- 将堆顶结点删除，将其值返回
- 将堆尾结点放到堆顶，并通过自顶向下堆化来修复堆

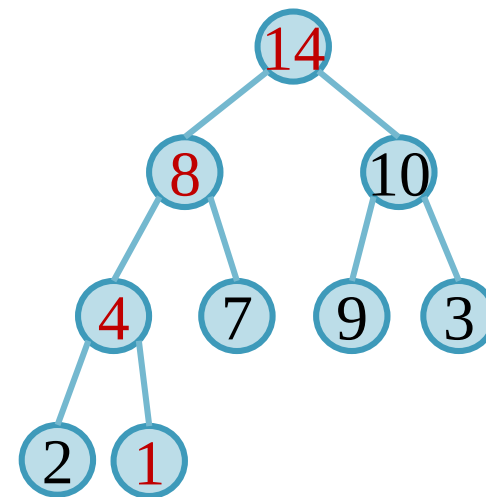
初始状态



堆尾结点置顶



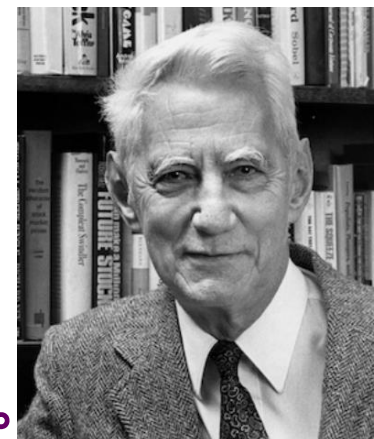
修复堆



# 本讲小结

- 二叉搜索树：隐含了一个线性有序序列
- 霍夫曼树与霍夫曼编码：思考从定性到定量描述的意义、体会对于编码问题用二叉树建模的妙处
- 堆：部分有序，让最大/最小元素露出来，堆化是其他操作的基础

信息论的创始人香农1952年在贝尔实验室的讲座中提到有助于创造性解决问题的6个思维技巧：简化，寻找类似问题，以不同形式复述问题，泛化，对问题进行结构化分析，反转问题。



加油同学们!

